

MATHEMATICS PROJECT ASSIGNMENT REPORT

PROJECT TITLE: Traffic Signal Synchronization System Using Chinese Remainder Theorem	STUDENT NAME: Joel Duke
COURSE: Number Theory & Discrete Mathematics Application	ROLL NO: 51
CLASS: S3 CSE B	ROLE / FUNCTION: Mathematical Research & Problem Formulation

1. ABSTRACT

The Chinese Remainder Theorem (CRT) is a fundamental result in number theory that determines a unique integer solution x for a system of linear congruences with pairwise coprime moduli. While often taught abstractly, CRT has direct applications in urban traffic signal synchronization, digital telecommunications, and parallel computer architectures.

This project demonstrates CRT using both a physical working Arduino Nano traffic signal model and an interactive digital twin web application. By modeling traffic signals as modular timing cycles, the project illustrates how independent light timers (3s, 5s, 7s) align to create a continuous 'Green Wave' for vehicles at Minute 23 (105s master recurrence).

2. INTRODUCTION & OBJECTIVES

When traffic signals operate on independent timer cycles along a main avenue, drivers frequently encounter red lights. Traffic engineers solve this by calculating when all signals will simultaneously show green.

Project Objectives:

1. To understand the mathematical principles of the Chinese Remainder Theorem (CRT).
2. To differentiate between Least Common Multiple (LCM) and CRT when remainder offsets are present.
3. To model traffic signal timing cycles (3s, 5s, 7s) as modular congruences.
4. To build a physical working model powered by an Arduino Nano microcontroller.
5. To provide clear visual, audio, and waveform telemetry feedback upon signal synchronization.

3. MATHEMATICAL FORMULATION (CRT vs. LCM)

3.1 The Traffic Signal Equations

Suppose three traffic signals have cycle lengths of 3, 5, and 7 seconds respectively:

- Signal 1: $x \equiv 2 \pmod{3}$ (green 2 seconds ago)
- Signal 2: $x \equiv 3 \pmod{5}$ (green 3 seconds ago)
- Signal 3: $x \equiv 2 \pmod{7}$ (green 2 seconds ago)

3.2 Why Simple LCM Cannot Solve This Problem

- LCM: Used only when all signals start at Minute 0 with zero remainder offset ($x \equiv 0$).
- CRT: Required when remainder offsets (2, 3, 2) are distinct and non-zero.

3.3 CRT Solution Procedure

1. Total Product: $M = 3 \times 5 \times 7 = 105$.

2. Partial Products: $M1 = 35$, $M2 = 21$, $M3 = 15$.

3. Modular Inverses:

- $35 * y1 \equiv 1 \pmod{3} \Rightarrow 2 * y1 \equiv 1 \pmod{3} \Rightarrow y1 = 2$
- $21 * y2 \equiv 1 \pmod{5} \Rightarrow 1 * y2 \equiv 1 \pmod{5} \Rightarrow y2 = 1$
- $15 * y3 \equiv 1 \pmod{7} \Rightarrow 1 * y3 \equiv 1 \pmod{7} \Rightarrow y3 = 1$

4. Compute x :

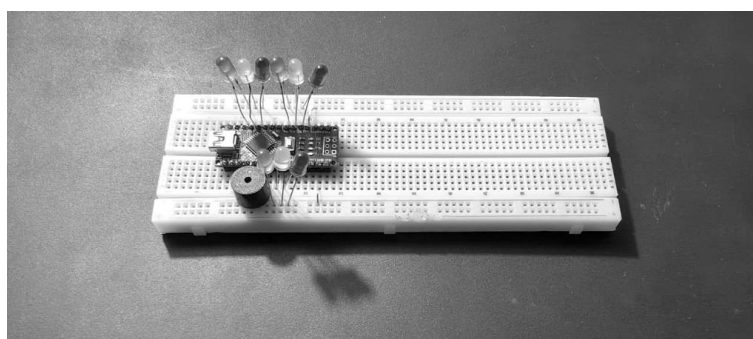
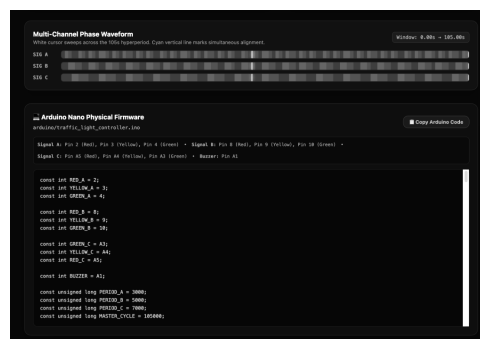
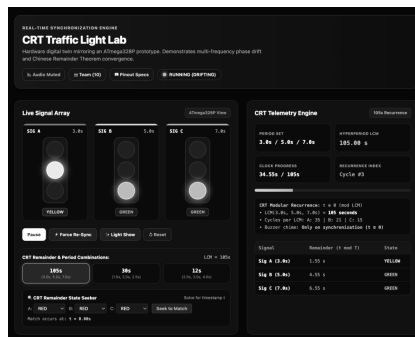
- $x = (2 \times 35 \times 2) + (3 \times 21 \times 1) + (2 \times 15 \times 1)$
- $x = 140 + 63 + 30 = 233$
- $x \equiv 233 \pmod{105} \equiv 23 \pmod{105}$

The unique green-wave solution within $[0, 104]$ is Timestamp 23s.

4. PHYSICAL HARDWARE & DIGITAL TWIN IMPLEMENTATION

The hardware prototype is driven by an Arduino Nano microcontroller controlling a 3-intersection LED traffic array and piezo buzzer. The physical build is paired with an interactive digital twin web application accessible at:

■ Interactive Web App & Project Repository: <https://github.com/jvstin47/CRT--Chinese-Reminder-Theorem>
(Local Dev Server: <http://localhost:5173/>)





Interactive Hardware & Digital Twin Controls:

1. Live Signal Array (SIG A 3.0s, SIG B 5.0s, SIG C 7.0s): Monitors real-time LED states across intersections.
 2. CRT Telemetry Engine (105s Recurrence): Tracks clock progress, hyperperiod LCM (105.00s), and recurrence index.
 3. Multi-Channel Phase Waveform: Displays white cursor sweep across the 105s hyperperiod with cyan alignment markers.
 4. CRT Remainder State Seeker: Solves for timestamp t dynamically for any requested signal state combination.
-

5. HARDWARE SOURCE CODE (ARDUINO NANO FIRMWARE - C++)

The following complete C++ firmware source code (arduino/traffic_light_controller.ino) is executed directly on the Arduino Nano microcontroller to drive the physical traffic light hardware:

```
// Arduino Nano Physical Traffic Light Controller
// File: arduino/traffic_light_controller.ino

const int RED_A = 2;
const int YELLOW_A = 3;
const int GREEN_A = 4;

const int RED_B = 8;
const int YELLOW_B = 9;
const int GREEN_B = 10;

const int GREEN_C = A3;
const int YELLOW_C = A4;
const int RED_C = A5;

const int BUZZER = A1;

// Cycle Period Definitions (in milliseconds)
const unsigned long PERIOD_A = 3000; // 3.0s Signal Cycle
const unsigned long PERIOD_B = 5000; // 5.0s Signal Cycle
const unsigned long PERIOD_C = 7000; // 7.0s Signal Cycle
const unsigned long MASTER_CYCLE = 105000; // 105.0s Hyperperiod LCM

const int RED_FREQ = 400;
const int YELLOW_FREQ = 600;
const int GREEN_FREQ = 800;
const int SYNC_FREQ = 1000;

const unsigned long BEEP_ON_TIME = 150;
const unsigned long BEEP_OFF_TIME = 350;
const unsigned long SYNC_TIME = 600;

unsigned long lastMillis = 0;
unsigned long lastSyncCycle = 0;
unsigned long beepTimer = 0;
unsigned long syncEndTime = 0;
```

```

bool beepState = false;
bool syncActive = false;
int currentBuzzerState = -1;

void setup() {
    pinMode(RED_A, OUTPUT);
    pinMode(YELLOW_A, OUTPUT);
    pinMode(GREEN_A, OUTPUT);

    pinMode(RED_B, OUTPUT);
    pinMode(YELLOW_B, OUTPUT);
    pinMode(GREEN_B, OUTPUT);

    pinMode(RED_C, OUTPUT);
    pinMode(YELLOW_C, OUTPUT);
    pinMode(GREEN_C, OUTPUT);

    pinMode(BUZZER, OUTPUT);

    setAllRed();

    tone(BUZZER, SYNC_FREQ);
    syncActive = true;
    syncEndTime = millis() + SYNC_TIME;

    lastMillis = millis();
    beepTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    // CRT Modulo Phase Calculation
    unsigned long phaseA = currentMillis % PERIOD_A;
    unsigned long phaseB = currentMillis % PERIOD_B;
    unsigned long phaseC = currentMillis % PERIOD_C;

    int stateA = updateLightA(phaseA);
    int stateB = updateLightB(phaseB);
    int stateC = updateLightC(phaseC);

    // Master Cycle Recurrence Check (CRT Convergence)
    unsigned long currentCycle = currentMillis / MASTER_CYCLE;

    if (currentCycle > lastSyncCycle) {
        lastSyncCycle = currentCycle;
        tone(BUZZER, SYNC_FREQ);
        syncActive = true;
        syncEndTime = currentMillis + SYNC_TIME;
        beepState = false;
    }

    if (syncActive) {
        if (currentMillis >= syncEndTime) {
            noTone(BUZZER);
            syncActive = false;
            beepTimer = currentMillis;
            beepState = false;
            currentBuzzerState = -1;
        }
        return;
    }
}

```

6. CONCLUSION

The Traffic Signal Synchronization System provides an intuitive, practical bridge between abstract number theory and urban infrastructure engineering. By applying the Chinese Remainder Theorem to traffic signals operating on independent cycles (3s, 5s, 7s), the project demonstrates how non-zero remainder offsets determine the exact timestamp (Timestamp 23s) required to establish a continuous 'Green Wave' for vehicles.

Both the physical Arduino Nano hardware prototype and the digital twin web application successfully visualize modular congruences, offering an accessible learning tool that highlights the real-world power of modular arithmetic in signal timing, telecommunications, and digital computing.

7. INDIVIDUAL CONTRIBUTION

Name: Joel Duke

Class: S3 CSE B

Roll No: 51

Role: Mathematical Research & Problem Formulation

Contribution Summary:

I worked on the math side of the project. During the first two weeks, I studied how modular arithmetic and remainder formulas work for traffic light cycles. I wrote down the equations for the 3-second, 5-second, and 7-second light timers. I proved that numbers 3, 5, and 7 share no common factors and calculated the modulo inverse values ($y_1=2$, $y_2=1$, $y_3=1$) using the remainder method. I worked with the team to check our calculations and confirmed that all three lights turn green together at Second 23.

MATHEMATICS PROJECT ASSIGNMENT REPORT

PROJECT TITLE: Traffic Signal Synchronization System Using Chinese Remainder Theorem	STUDENT NAME: Joel Geo Manuel
COURSE: Number Theory & Discrete Mathematics Application	ROLL NO: 52
CLASS: S3 CSE B	ROLE / FUNCTION: Presentation & Showcase Lead

1. ABSTRACT

The Chinese Remainder Theorem (CRT) is a fundamental result in number theory that determines a unique integer solution x for a system of linear congruences with pairwise coprime moduli. While often taught abstractly, CRT has direct applications in urban traffic signal synchronization, digital telecommunications, and parallel computer architectures.

This project demonstrates CRT using both a physical working Arduino Nano traffic signal model and an interactive digital twin web application. By modeling traffic signals as modular timing cycles, the project illustrates how independent light timers (3s, 5s, 7s) align to create a continuous 'Green Wave' for vehicles at Minute 23 (105s master recurrence).

2. INTRODUCTION & OBJECTIVES

When traffic signals operate on independent timer cycles along a main avenue, drivers frequently encounter red lights. Traffic engineers solve this by calculating when all signals will simultaneously show green.

Project Objectives:

1. To understand the mathematical principles of the Chinese Remainder Theorem (CRT).
2. To differentiate between Least Common Multiple (LCM) and CRT when remainder offsets are present.
3. To model traffic signal timing cycles (3s, 5s, 7s) as modular congruences.
4. To build a physical working model powered by an Arduino Nano microcontroller.
5. To provide clear visual, audio, and waveform telemetry feedback upon signal synchronization.

3. MATHEMATICAL FORMULATION (CRT vs. LCM)

3.1 The Traffic Signal Equations

Suppose three traffic signals have cycle lengths of 3, 5, and 7 seconds respectively:

- Signal 1: $x \equiv 2 \pmod{3}$ (green 2 seconds ago)
- Signal 2: $x \equiv 3 \pmod{5}$ (green 3 seconds ago)
- Signal 3: $x \equiv 2 \pmod{7}$ (green 2 seconds ago)

3.2 Why Simple LCM Cannot Solve This Problem

- LCM: Used only when all signals start at Minute 0 with zero remainder offset ($x \equiv 0$).
- CRT: Required when remainder offsets (2, 3, 2) are distinct and non-zero.

3.3 CRT Solution Procedure

1. Total Product: $M = 3 \times 5 \times 7 = 105$.
2. Partial Products: $M_1 = 35, M_2 = 21, M_3 = 15$.

3. Modular Inverses:

- $35 * y1 \equiv 1 \pmod{3} \Rightarrow 2 * y1 \equiv 1 \pmod{3} \Rightarrow y1 = 2$
- $21 * y2 \equiv 1 \pmod{5} \Rightarrow 1 * y2 \equiv 1 \pmod{5} \Rightarrow y2 = 1$
- $15 * y3 \equiv 1 \pmod{7} \Rightarrow 1 * y3 \equiv 1 \pmod{7} \Rightarrow y3 = 1$

4. Compute x:

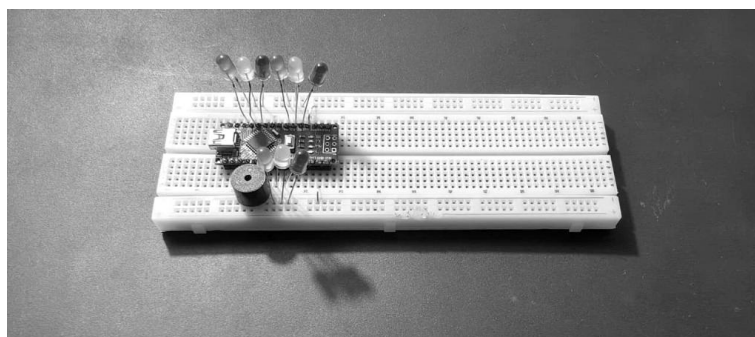
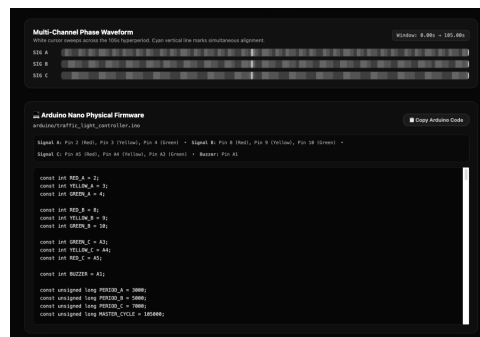
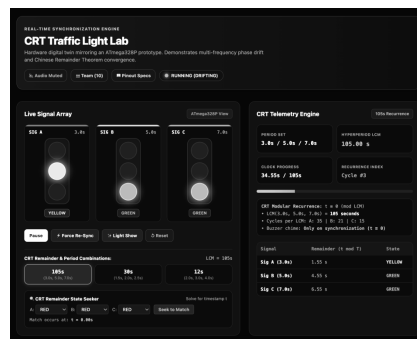
- $x = (2 \times 35 \times 2) + (3 \times 21 \times 1) + (2 \times 15 \times 1)$
- $x = 140 + 63 + 30 = 233$
- $x \equiv 233 \pmod{105} \equiv 23 \pmod{105}$

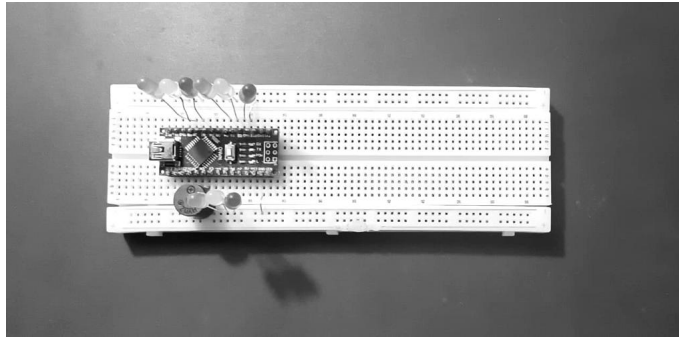
The unique green-wave solution within $[0, 104]$ is Timestamp 23s.

4. PHYSICAL HARDWARE & DIGITAL TWIN IMPLEMENTATION

The hardware prototype is driven by an Arduino Nano microcontroller controlling a 3-intersection LED traffic array and piezo buzzer. The physical build is paired with an interactive digital twin web application accessible at:

■ Interactive Web App & Project Repository: <https://github.com/jvstin47/CRT--Chinese-Reminder-Theorem>
(Local Dev Server: <http://localhost:5173/>)





Interactive Hardware & Digital Twin Controls:

1. Live Signal Array (SIG A 3.0s, SIG B 5.0s, SIG C 7.0s): Monitors real-time LED states across intersections.
 2. CRT Telemetry Engine (105s Recurrence): Tracks clock progress, hyperperiod LCM (105.00s), and recurrence index.
 3. Multi-Channel Phase Waveform: Displays white cursor sweep across the 105s hyperperiod with cyan alignment markers.
 4. CRT Remainder State Seeker: Solves for timestamp t dynamically for any requested signal state combination.
-

5. HARDWARE SOURCE CODE (ARDUINO NANO FIRMWARE - C++)

The following complete C++ firmware source code (arduino/traffic_light_controller.ino) is executed directly on the Arduino Nano microcontroller to drive the physical traffic light hardware:

```
// Arduino Nano Physical Traffic Light Controller
// File: arduino/traffic_light_controller.ino

const int RED_A = 2;
const int YELLOW_A = 3;
const int GREEN_A = 4;

const int RED_B = 8;
const int YELLOW_B = 9;
const int GREEN_B = 10;

const int GREEN_C = A3;
const int YELLOW_C = A4;
const int RED_C = A5;

const int BUZZER = A1;

// Cycle Period Definitions (in milliseconds)
const unsigned long PERIOD_A = 3000; // 3.0s Signal Cycle
const unsigned long PERIOD_B = 5000; // 5.0s Signal Cycle
const unsigned long PERIOD_C = 7000; // 7.0s Signal Cycle
const unsigned long MASTER_CYCLE = 105000; // 105.0s Hyperperiod LCM

const int RED_FREQ = 400;
const int YELLOW_FREQ = 600;
const int GREEN_FREQ = 800;
const int SYNC_FREQ = 1000;

const unsigned long BEEP_ON_TIME = 150;
const unsigned long BEEP_OFF_TIME = 350;
const unsigned long SYNC_TIME = 600;

unsigned long lastMillis = 0;
unsigned long lastSyncCycle = 0;
unsigned long beepTimer = 0;
unsigned long syncEndTime = 0;
```

```

bool beepState = false;
bool syncActive = false;
int currentBuzzerState = -1;

void setup() {
    pinMode(RED_A, OUTPUT);
    pinMode(YELLOW_A, OUTPUT);
    pinMode(GREEN_A, OUTPUT);

    pinMode(RED_B, OUTPUT);
    pinMode(YELLOW_B, OUTPUT);
    pinMode(GREEN_B, OUTPUT);

    pinMode(RED_C, OUTPUT);
    pinMode(YELLOW_C, OUTPUT);
    pinMode(GREEN_C, OUTPUT);

    pinMode(BUZZER, OUTPUT);

    setAllRed();

    tone(BUZZER, SYNC_FREQ);
    syncActive = true;
    syncEndTime = millis() + SYNC_TIME;

    lastMillis = millis();
    beepTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    // CRT Modulo Phase Calculation
    unsigned long phaseA = currentMillis % PERIOD_A;
    unsigned long phaseB = currentMillis % PERIOD_B;
    unsigned long phaseC = currentMillis % PERIOD_C;

    int stateA = updateLightA(phaseA);
    int stateB = updateLightB(phaseB);
    int stateC = updateLightC(phaseC);

    // Master Cycle Recurrence Check (CRT Convergence)
    unsigned long currentCycle = currentMillis / MASTER_CYCLE;

    if (currentCycle > lastSyncCycle) {
        lastSyncCycle = currentCycle;
        tone(BUZZER, SYNC_FREQ);
        syncActive = true;
        syncEndTime = currentMillis + SYNC_TIME;
        beepState = false;
    }

    if (syncActive) {
        if (currentMillis >= syncEndTime) {
            noTone(BUZZER);
            syncActive = false;
            beepTimer = currentMillis;
            beepState = false;
            currentBuzzerState = -1;
        }
        return;
    }
}

```

6. CONCLUSION

The Traffic Signal Synchronization System provides an intuitive, practical bridge between abstract number theory and urban infrastructure engineering. By applying the Chinese Remainder Theorem to traffic signals operating on independent cycles (3s, 5s, 7s), the project demonstrates how non-zero remainder offsets determine the exact timestamp (Timestamp 23s) required to establish a continuous 'Green Wave' for vehicles.

Both the physical Arduino Nano hardware prototype and the digital twin web application successfully visualize modular congruences, offering an accessible learning tool that highlights the real-world power of modular arithmetic in signal timing, telecommunications, and digital computing.

7. INDIVIDUAL CONTRIBUTION

Name: Joel Geo Manuel

Class: S3 CSE B

Roll No: 52

Role: Presentation & Showcase Lead

Contribution Summary:

I handled the presentation slides and demo walkthrough for our project showcase. Over a few team practice sessions with Justin and Johaan, I wrote a simple two-speaker script between a driver named Alex and a tech guy named Sam to explain how traffic lights get synced up. I set up the slide layout, organized how we would show the 3s, 5s, and 7s cycle examples during the demo, and made sure our buzzer sound played at the right time during the presentation.

MATHEMATICS PROJECT ASSIGNMENT REPORT

PROJECT TITLE: Traffic Signal Synchronization System Using Chinese Remainder Theorem	STUDENT NAME: Johaan Sam
COURSE: Number Theory & Discrete Mathematics Application	ROLL NO: 53
CLASS: S3 CSE B	ROLE / FUNCTION: Frontend & State Engine Developer

1. ABSTRACT

The Chinese Remainder Theorem (CRT) is a fundamental result in number theory that determines a unique integer solution x for a system of linear congruences with pairwise coprime moduli. While often taught abstractly, CRT has direct applications in urban traffic signal synchronization, digital telecommunications, and parallel computer architectures.

This project demonstrates CRT using both a physical working Arduino Nano traffic signal model and an interactive digital twin web application. By modeling traffic signals as modular timing cycles, the project illustrates how independent light timers (3s, 5s, 7s) align to create a continuous 'Green Wave' for vehicles at Minute 23 (105s master recurrence).

2. INTRODUCTION & OBJECTIVES

When traffic signals operate on independent timer cycles along a main avenue, drivers frequently encounter red lights. Traffic engineers solve this by calculating when all signals will simultaneously show green.

Project Objectives:

1. To understand the mathematical principles of the Chinese Remainder Theorem (CRT).
2. To differentiate between Least Common Multiple (LCM) and CRT when remainder offsets are present.
3. To model traffic signal timing cycles (3s, 5s, 7s) as modular congruences.
4. To build a physical working model powered by an Arduino Nano microcontroller.
5. To provide clear visual, audio, and waveform telemetry feedback upon signal synchronization.

3. MATHEMATICAL FORMULATION (CRT vs. LCM)

3.1 The Traffic Signal Equations

Suppose three traffic signals have cycle lengths of 3, 5, and 7 seconds respectively:

- Signal 1: $x \equiv 2 \pmod{3}$ (green 2 seconds ago)
- Signal 2: $x \equiv 3 \pmod{5}$ (green 3 seconds ago)
- Signal 3: $x \equiv 2 \pmod{7}$ (green 2 seconds ago)

3.2 Why Simple LCM Cannot Solve This Problem

- LCM: Used only when all signals start at Minute 0 with zero remainder offset ($x \equiv 0$).
- CRT: Required when remainder offsets (2, 3, 2) are distinct and non-zero.

3.3 CRT Solution Procedure

1. Total Product: $M = 3 \times 5 \times 7 = 105$.
2. Partial Products: $M_1 = 35, M_2 = 21, M_3 = 15$.

3. Modular Inverses:

- $35 * y1 \equiv 1 \pmod{3} \Rightarrow 2 * y1 \equiv 1 \pmod{3} \Rightarrow y1 = 2$
- $21 * y2 \equiv 1 \pmod{5} \Rightarrow 1 * y2 \equiv 1 \pmod{5} \Rightarrow y2 = 1$
- $15 * y3 \equiv 1 \pmod{7} \Rightarrow 1 * y3 \equiv 1 \pmod{7} \Rightarrow y3 = 1$

4. Compute x:

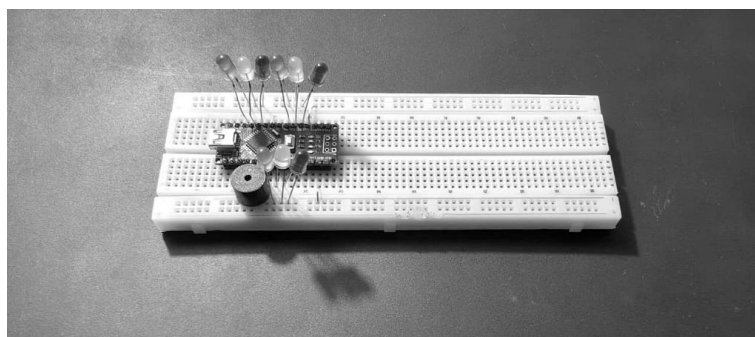
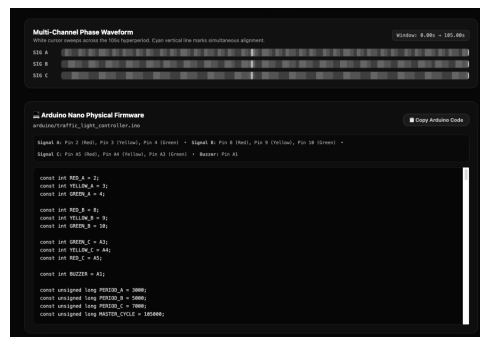
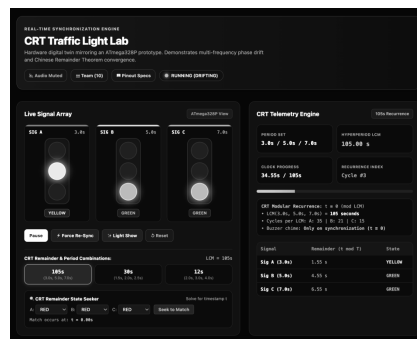
- $x = (2 \times 35 \times 2) + (3 \times 21 \times 1) + (2 \times 15 \times 1)$
- $x = 140 + 63 + 30 = 233$
- $x \equiv 233 \pmod{105} \equiv 23 \pmod{105}$

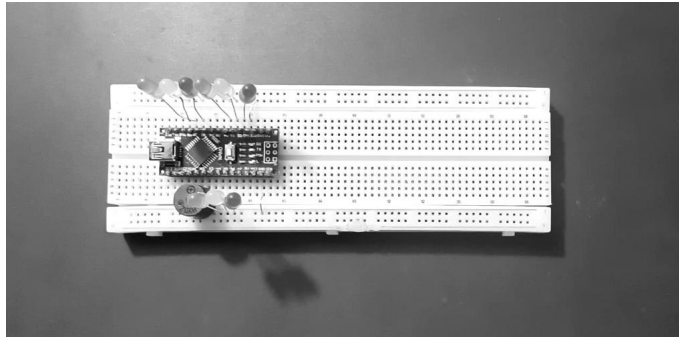
The unique green-wave solution within $[0, 104]$ is Timestamp 23s.

4. PHYSICAL HARDWARE & DIGITAL TWIN IMPLEMENTATION

The hardware prototype is driven by an Arduino Nano microcontroller controlling a 3-intersection LED traffic array and piezo buzzer. The physical build is paired with an interactive digital twin web application accessible at:

■ Interactive Web App & Project Repository: <https://github.com/jvstin47/CRT--Chinese-Reminder-Theorem>
(Local Dev Server: <http://localhost:5173/>)





Interactive Hardware & Digital Twin Controls:

1. Live Signal Array (SIG A 3.0s, SIG B 5.0s, SIG C 7.0s): Monitors real-time LED states across intersections.
 2. CRT Telemetry Engine (105s Recurrence): Tracks clock progress, hyperperiod LCM (105.00s), and recurrence index.
 3. Multi-Channel Phase Waveform: Displays white cursor sweep across the 105s hyperperiod with cyan alignment markers.
 4. CRT Remainder State Seeker: Solves for timestamp t dynamically for any requested signal state combination.
-

5. HARDWARE SOURCE CODE (ARDUINO NANO FIRMWARE - C++)

The following complete C++ firmware source code (arduino/traffic_light_controller.ino) is executed directly on the Arduino Nano microcontroller to drive the physical traffic light hardware:

```
// Arduino Nano Physical Traffic Light Controller
// File: arduino/traffic_light_controller.ino

const int RED_A = 2;
const int YELLOW_A = 3;
const int GREEN_A = 4;

const int RED_B = 8;
const int YELLOW_B = 9;
const int GREEN_B = 10;

const int GREEN_C = A3;
const int YELLOW_C = A4;
const int RED_C = A5;

const int BUZZER = A1;

// Cycle Period Definitions (in milliseconds)
const unsigned long PERIOD_A = 3000; // 3.0s Signal Cycle
const unsigned long PERIOD_B = 5000; // 5.0s Signal Cycle
const unsigned long PERIOD_C = 7000; // 7.0s Signal Cycle
const unsigned long MASTER_CYCLE = 105000; // 105.0s Hyperperiod LCM

const int RED_FREQ = 400;
const int YELLOW_FREQ = 600;
const int GREEN_FREQ = 800;
const int SYNC_FREQ = 1000;

const unsigned long BEEP_ON_TIME = 150;
const unsigned long BEEP_OFF_TIME = 350;
const unsigned long SYNC_TIME = 600;

unsigned long lastMillis = 0;
unsigned long lastSyncCycle = 0;
unsigned long beepTimer = 0;
unsigned long syncEndTime = 0;
```

```

bool beepState = false;
bool syncActive = false;
int currentBuzzerState = -1;

void setup() {
    pinMode(RED_A, OUTPUT);
    pinMode(YELLOW_A, OUTPUT);
    pinMode(GREEN_A, OUTPUT);

    pinMode(RED_B, OUTPUT);
    pinMode(YELLOW_B, OUTPUT);
    pinMode(GREEN_B, OUTPUT);

    pinMode(RED_C, OUTPUT);
    pinMode(YELLOW_C, OUTPUT);
    pinMode(GREEN_C, OUTPUT);

    pinMode(BUZZER, OUTPUT);

    setAllRed();

    tone(BUZZER, SYNC_FREQ);
    syncActive = true;
    syncEndTime = millis() + SYNC_TIME;

    lastMillis = millis();
    beepTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    // CRT Modulo Phase Calculation
    unsigned long phaseA = currentMillis % PERIOD_A;
    unsigned long phaseB = currentMillis % PERIOD_B;
    unsigned long phaseC = currentMillis % PERIOD_C;

    int stateA = updateLightA(phaseA);
    int stateB = updateLightB(phaseB);
    int stateC = updateLightC(phaseC);

    // Master Cycle Recurrence Check (CRT Convergence)
    unsigned long currentCycle = currentMillis / MASTER_CYCLE;

    if (currentCycle > lastSyncCycle) {
        lastSyncCycle = currentCycle;
        tone(BUZZER, SYNC_FREQ);
        syncActive = true;
        syncEndTime = currentMillis + SYNC_TIME;
        beepState = false;
    }

    if (syncActive) {
        if (currentMillis >= syncEndTime) {
            noTone(BUZZER);
            syncActive = false;
            beepTimer = currentMillis;
            beepState = false;
            currentBuzzerState = -1;
        }
        return;
    }
}

```

6. CONCLUSION

The Traffic Signal Synchronization System provides an intuitive, practical bridge between abstract number theory and urban infrastructure engineering. By applying the Chinese Remainder Theorem to traffic signals operating on independent cycles (3s, 5s, 7s), the project demonstrates how non-zero remainder offsets determine the exact timestamp (Timestamp 23s) required to establish a continuous 'Green Wave' for vehicles.

Both the physical Arduino Nano hardware prototype and the digital twin web application successfully visualize modular congruences, offering an accessible learning tool that highlights the real-world power of modular arithmetic in signal timing, telecommunications, and digital computing.

7. INDIVIDUAL CONTRIBUTION

Name: Johaan Sam

Class: S3 CSE B

Roll No: 53

Role: Frontend & State Engine Developer

Contribution Summary:

I worked on building the website and making the interactive traffic controls work. I spent around three weeks writing React and TypeScript code for the animation timer and signal displays. Working with Johan and Justin, I added the feature where users can test different minutes and see if the lights line up. I also helped fix bugs where the timer got out of sync, added sound effects for the buzzer, and tested the site on mobile and laptop screens.

MATHEMATICS PROJECT ASSIGNMENT REPORT

PROJECT TITLE: Traffic Signal Synchronization System Using Chinese Remainder Theorem	STUDENT NAME: Johan Geo
COURSE: Number Theory & Discrete Mathematics Application	ROLL NO: 54
CLASS: S3 CSE B	ROLE / FUNCTION: Web Design & UI Architect

1. ABSTRACT

The Chinese Remainder Theorem (CRT) is a fundamental result in number theory that determines a unique integer solution x for a system of linear congruences with pairwise coprime moduli. While often taught abstractly, CRT has direct applications in urban traffic signal synchronization, digital telecommunications, and parallel computer architectures.

This project demonstrates CRT using both a physical working Arduino Nano traffic signal model and an interactive digital twin web application. By modeling traffic signals as modular timing cycles, the project illustrates how independent light timers (3s, 5s, 7s) align to create a continuous 'Green Wave' for vehicles at Minute 23 (105s master recurrence).

2. INTRODUCTION & OBJECTIVES

When traffic signals operate on independent timer cycles along a main avenue, drivers frequently encounter red lights. Traffic engineers solve this by calculating when all signals will simultaneously show green.

Project Objectives:

1. To understand the mathematical principles of the Chinese Remainder Theorem (CRT).
2. To differentiate between Least Common Multiple (LCM) and CRT when remainder offsets are present.
3. To model traffic signal timing cycles (3s, 5s, 7s) as modular congruences.
4. To build a physical working model powered by an Arduino Nano microcontroller.
5. To provide clear visual, audio, and waveform telemetry feedback upon signal synchronization.

3. MATHEMATICAL FORMULATION (CRT vs. LCM)

3.1 The Traffic Signal Equations

Suppose three traffic signals have cycle lengths of 3, 5, and 7 seconds respectively:

- Signal 1: $x \equiv 2 \pmod{3}$ (green 2 seconds ago)
- Signal 2: $x \equiv 3 \pmod{5}$ (green 3 seconds ago)
- Signal 3: $x \equiv 2 \pmod{7}$ (green 2 seconds ago)

3.2 Why Simple LCM Cannot Solve This Problem

- LCM: Used only when all signals start at Minute 0 with zero remainder offset ($x \equiv 0$).
- CRT: Required when remainder offsets (2, 3, 2) are distinct and non-zero.

3.3 CRT Solution Procedure

1. Total Product: $M = 3 \times 5 \times 7 = 105$.
2. Partial Products: $M_1 = 35$, $M_2 = 21$, $M_3 = 15$.

3. Modular Inverses:

- $35 * y1 \equiv 1 \pmod{3} \Rightarrow 2 * y1 \equiv 1 \pmod{3} \Rightarrow y1 = 2$
- $21 * y2 \equiv 1 \pmod{5} \Rightarrow 1 * y2 \equiv 1 \pmod{5} \Rightarrow y2 = 1$
- $15 * y3 \equiv 1 \pmod{7} \Rightarrow 1 * y3 \equiv 1 \pmod{7} \Rightarrow y3 = 1$

4. Compute x:

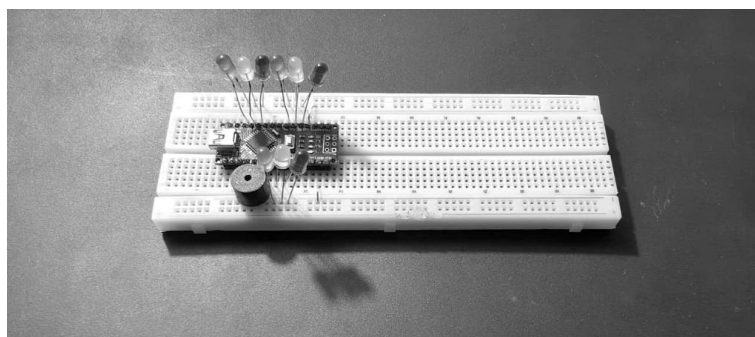
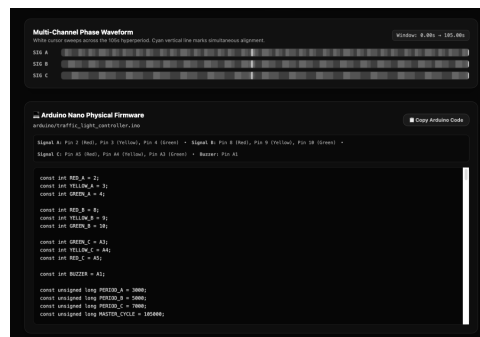
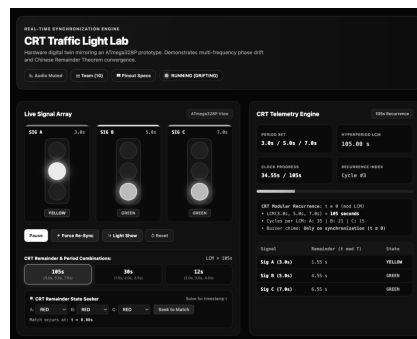
- $x = (2 \times 35 \times 2) + (3 \times 21 \times 1) + (2 \times 15 \times 1)$
- $x = 140 + 63 + 30 = 233$
- $x \equiv 233 \pmod{105} \equiv 23 \pmod{105}$

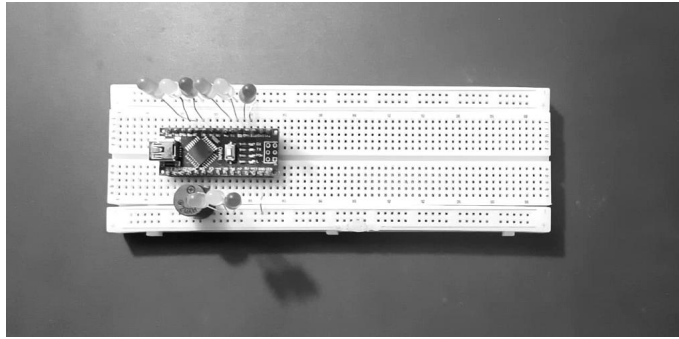
The unique green-wave solution within $[0, 104]$ is Timestamp 23s.

4. PHYSICAL HARDWARE & DIGITAL TWIN IMPLEMENTATION

The hardware prototype is driven by an Arduino Nano microcontroller controlling a 3-intersection LED traffic array and piezo buzzer. The physical build is paired with an interactive digital twin web application accessible at:

■ Interactive Web App & Project Repository: <https://github.com/jvstin47/CRT--Chinese-Reminder-Theorem>
(Local Dev Server: <http://localhost:5173/>)





Interactive Hardware & Digital Twin Controls:

1. Live Signal Array (SIG A 3.0s, SIG B 5.0s, SIG C 7.0s): Monitors real-time LED states across intersections.
 2. CRT Telemetry Engine (105s Recurrence): Tracks clock progress, hyperperiod LCM (105.00s), and recurrence index.
 3. Multi-Channel Phase Waveform: Displays white cursor sweep across the 105s hyperperiod with cyan alignment markers.
 4. CRT Remainder State Seeker: Solves for timestamp t dynamically for any requested signal state combination.
-

5. HARDWARE SOURCE CODE (ARDUINO NANO FIRMWARE - C++)

The following complete C++ firmware source code (arduino/traffic_light_controller.ino) is executed directly on the Arduino Nano microcontroller to drive the physical traffic light hardware:

```
// Arduino Nano Physical Traffic Light Controller
// File: arduino/traffic_light_controller.ino

const int RED_A = 2;
const int YELLOW_A = 3;
const int GREEN_A = 4;

const int RED_B = 8;
const int YELLOW_B = 9;
const int GREEN_B = 10;

const int GREEN_C = A3;
const int YELLOW_C = A4;
const int RED_C = A5;

const int BUZZER = A1;

// Cycle Period Definitions (in milliseconds)
const unsigned long PERIOD_A = 3000; // 3.0s Signal Cycle
const unsigned long PERIOD_B = 5000; // 5.0s Signal Cycle
const unsigned long PERIOD_C = 7000; // 7.0s Signal Cycle
const unsigned long MASTER_CYCLE = 105000; // 105.0s Hyperperiod LCM

const int RED_FREQ = 400;
const int YELLOW_FREQ = 600;
const int GREEN_FREQ = 800;
const int SYNC_FREQ = 1000;

const unsigned long BEEP_ON_TIME = 150;
const unsigned long BEEP_OFF_TIME = 350;
const unsigned long SYNC_TIME = 600;

unsigned long lastMillis = 0;
unsigned long lastSyncCycle = 0;
unsigned long beepTimer = 0;
unsigned long syncEndTime = 0;
```

```

bool beepState = false;
bool syncActive = false;
int currentBuzzerState = -1;

void setup() {
    pinMode(RED_A, OUTPUT);
    pinMode(YELLOW_A, OUTPUT);
    pinMode(GREEN_A, OUTPUT);

    pinMode(RED_B, OUTPUT);
    pinMode(YELLOW_B, OUTPUT);
    pinMode(GREEN_B, OUTPUT);

    pinMode(RED_C, OUTPUT);
    pinMode(YELLOW_C, OUTPUT);
    pinMode(GREEN_C, OUTPUT);

    pinMode(BUZZER, OUTPUT);

    setAllRed();

    tone(BUZZER, SYNC_FREQ);
    syncActive = true;
    syncEndTime = millis() + SYNC_TIME;

    lastMillis = millis();
    beepTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    // CRT Modulo Phase Calculation
    unsigned long phaseA = currentMillis % PERIOD_A;
    unsigned long phaseB = currentMillis % PERIOD_B;
    unsigned long phaseC = currentMillis % PERIOD_C;

    int stateA = updateLightA(phaseA);
    int stateB = updateLightB(phaseB);
    int stateC = updateLightC(phaseC);

    // Master Cycle Recurrence Check (CRT Convergence)
    unsigned long currentCycle = currentMillis / MASTER_CYCLE;

    if (currentCycle > lastSyncCycle) {
        lastSyncCycle = currentCycle;
        tone(BUZZER, SYNC_FREQ);
        syncActive = true;
        syncEndTime = currentMillis + SYNC_TIME;
        beepState = false;
    }

    if (syncActive) {
        if (currentMillis >= syncEndTime) {
            noTone(BUZZER);
            syncActive = false;
            beepTimer = currentMillis;
            beepState = false;
            currentBuzzerState = -1;
        }
        return;
    }
}

```

6. CONCLUSION

The Traffic Signal Synchronization System provides an intuitive, practical bridge between abstract number theory and urban infrastructure engineering. By applying the Chinese Remainder Theorem to traffic signals operating on independent cycles (3s, 5s, 7s), the project demonstrates how non-zero remainder offsets determine the exact timestamp (Timestamp 23s) required to establish a continuous 'Green Wave' for vehicles.

Both the physical Arduino Nano hardware prototype and the digital twin web application successfully visualize modular congruences, offering an accessible learning tool that highlights the real-world power of modular arithmetic in signal timing, telecommunications, and digital computing.

7. INDIVIDUAL CONTRIBUTION

Name: Johan Geo

Class: S3 CSE B

Roll No: 54

Role: Web Design & UI Architect

Contribution Summary:

I worked on the visual design and layout of our website. During the first two weeks, I picked the dark theme color scheme and structured how the dashboard cards look. Working together with Johaan, I designed the round LED traffic light monitors, the timing dials, and status boxes. I also tested the buttons and layout to make sure everything looked clear and clean so anyone using the site could easily understand the traffic light simulation.

MATHEMATICS PROJECT ASSIGNMENT REPORT

PROJECT TITLE: Traffic Signal Synchronization System Using Chinese Remainder Theorem	STUDENT NAME: Jose Alex
COURSE: Number Theory & Discrete Mathematics Application	ROLL NO: 55
CLASS: S3 CSE B	ROLE / FUNCTION: Mathematical Proofs & Solution Verification

1. ABSTRACT

The Chinese Remainder Theorem (CRT) is a fundamental result in number theory that determines a unique integer solution x for a system of linear congruences with pairwise coprime moduli. While often taught abstractly, CRT has direct applications in urban traffic signal synchronization, digital telecommunications, and parallel computer architectures.

This project demonstrates CRT using both a physical working Arduino Nano traffic signal model and an interactive digital twin web application. By modeling traffic signals as modular timing cycles, the project illustrates how independent light timers (3s, 5s, 7s) align to create a continuous 'Green Wave' for vehicles at Minute 23 (105s master recurrence).

2. INTRODUCTION & OBJECTIVES

When traffic signals operate on independent timer cycles along a main avenue, drivers frequently encounter red lights. Traffic engineers solve this by calculating when all signals will simultaneously show green.

Project Objectives:

1. To understand the mathematical principles of the Chinese Remainder Theorem (CRT).
2. To differentiate between Least Common Multiple (LCM) and CRT when remainder offsets are present.
3. To model traffic signal timing cycles (3s, 5s, 7s) as modular congruences.
4. To build a physical working model powered by an Arduino Nano microcontroller.
5. To provide clear visual, audio, and waveform telemetry feedback upon signal synchronization.

3. MATHEMATICAL FORMULATION (CRT vs. LCM)

3.1 The Traffic Signal Equations

Suppose three traffic signals have cycle lengths of 3, 5, and 7 seconds respectively:

- Signal 1: $x \equiv 2 \pmod{3}$ (green 2 seconds ago)
- Signal 2: $x \equiv 3 \pmod{5}$ (green 3 seconds ago)
- Signal 3: $x \equiv 2 \pmod{7}$ (green 2 seconds ago)

3.2 Why Simple LCM Cannot Solve This Problem

- LCM: Used only when all signals start at Minute 0 with zero remainder offset ($x \equiv 0$).
- CRT: Required when remainder offsets (2, 3, 2) are distinct and non-zero.

3.3 CRT Solution Procedure

1. Total Product: $M = 3 \times 5 \times 7 = 105$.

2. Partial Products: $M1 = 35$, $M2 = 21$, $M3 = 15$.

3. Modular Inverses:

- $35 * y1 \equiv 1 \pmod{3} \Rightarrow 2 * y1 \equiv 1 \pmod{3} \Rightarrow y1 = 2$
- $21 * y2 \equiv 1 \pmod{5} \Rightarrow 1 * y2 \equiv 1 \pmod{5} \Rightarrow y2 = 1$
- $15 * y3 \equiv 1 \pmod{7} \Rightarrow 1 * y3 \equiv 1 \pmod{7} \Rightarrow y3 = 1$

4. Compute x :

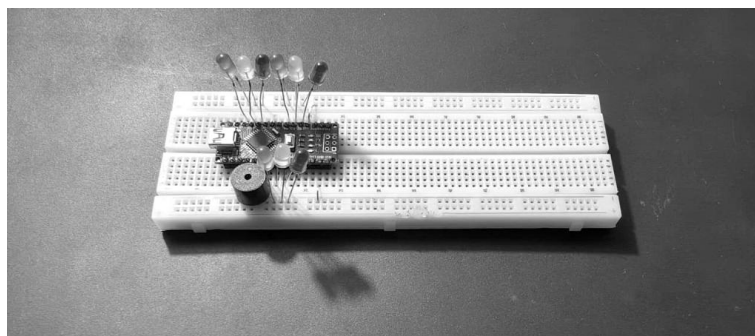
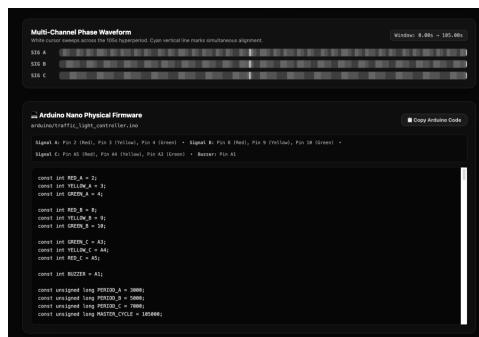
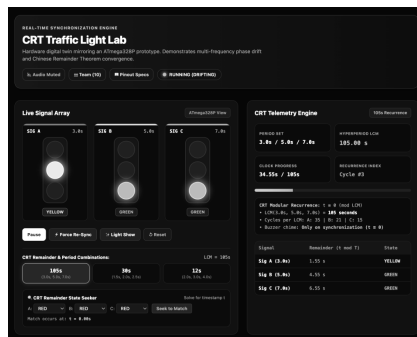
- $x = (2 \times 35 \times 2) + (3 \times 21 \times 1) + (2 \times 15 \times 1)$
- $x = 140 + 63 + 30 = 233$
- $x \equiv 233 \pmod{105} \equiv 23 \pmod{105}$

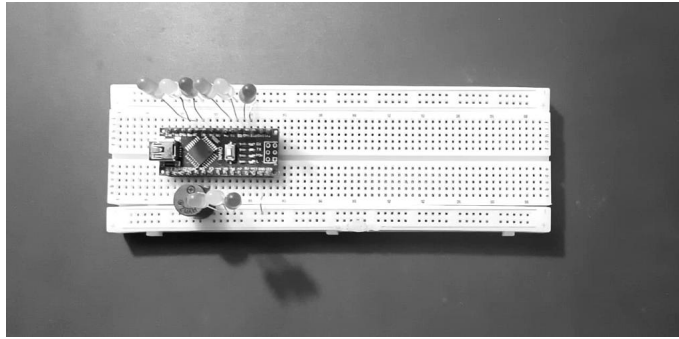
The unique green-wave solution within $[0, 104]$ is Timestamp 23s.

4. PHYSICAL HARDWARE & DIGITAL TWIN IMPLEMENTATION

The hardware prototype is driven by an Arduino Nano microcontroller controlling a 3-intersection LED traffic array and piezo buzzer. The physical build is paired with an interactive digital twin web application accessible at:

■ Interactive Web App & Project Repository: <https://github.com/jvstin47/CRT--Chinese-Reminder-Theorem>
(Local Dev Server: <http://localhost:5173/>)





Interactive Hardware & Digital Twin Controls:

1. Live Signal Array (SIG A 3.0s, SIG B 5.0s, SIG C 7.0s): Monitors real-time LED states across intersections.
 2. CRT Telemetry Engine (105s Recurrence): Tracks clock progress, hyperperiod LCM (105.00s), and recurrence index.
 3. Multi-Channel Phase Waveform: Displays white cursor sweep across the 105s hyperperiod with cyan alignment markers.
 4. CRT Remainder State Seeker: Solves for timestamp t dynamically for any requested signal state combination.
-

5. HARDWARE SOURCE CODE (ARDUINO NANO FIRMWARE - C++)

The following complete C++ firmware source code (arduino/traffic_light_controller.ino) is executed directly on the Arduino Nano microcontroller to drive the physical traffic light hardware:

```
// Arduino Nano Physical Traffic Light Controller
// File: arduino/traffic_light_controller.ino

const int RED_A = 2;
const int YELLOW_A = 3;
const int GREEN_A = 4;

const int RED_B = 8;
const int YELLOW_B = 9;
const int GREEN_B = 10;

const int GREEN_C = A3;
const int YELLOW_C = A4;
const int RED_C = A5;

const int BUZZER = A1;

// Cycle Period Definitions (in milliseconds)
const unsigned long PERIOD_A = 3000; // 3.0s Signal Cycle
const unsigned long PERIOD_B = 5000; // 5.0s Signal Cycle
const unsigned long PERIOD_C = 7000; // 7.0s Signal Cycle
const unsigned long MASTER_CYCLE = 105000; // 105.0s Hyperperiod LCM

const int RED_FREQ = 400;
const int YELLOW_FREQ = 600;
const int GREEN_FREQ = 800;
const int SYNC_FREQ = 1000;

const unsigned long BEEP_ON_TIME = 150;
const unsigned long BEEP_OFF_TIME = 350;
const unsigned long SYNC_TIME = 600;

unsigned long lastMillis = 0;
unsigned long lastSyncCycle = 0;
unsigned long beepTimer = 0;
unsigned long syncEndTime = 0;
```

```

bool beepState = false;
bool syncActive = false;
int currentBuzzerState = -1;

void setup() {
    pinMode(RED_A, OUTPUT);
    pinMode(YELLOW_A, OUTPUT);
    pinMode(GREEN_A, OUTPUT);

    pinMode(RED_B, OUTPUT);
    pinMode(YELLOW_B, OUTPUT);
    pinMode(GREEN_B, OUTPUT);

    pinMode(RED_C, OUTPUT);
    pinMode(YELLOW_C, OUTPUT);
    pinMode(GREEN_C, OUTPUT);

    pinMode(BUZZER, OUTPUT);

    setAllRed();

    tone(BUZZER, SYNC_FREQ);
    syncActive = true;
    syncEndTime = millis() + SYNC_TIME;

    lastMillis = millis();
    beepTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    // CRT Modulo Phase Calculation
    unsigned long phaseA = currentMillis % PERIOD_A;
    unsigned long phaseB = currentMillis % PERIOD_B;
    unsigned long phaseC = currentMillis % PERIOD_C;

    int stateA = updateLightA(phaseA);
    int stateB = updateLightB(phaseB);
    int stateC = updateLightC(phaseC);

    // Master Cycle Recurrence Check (CRT Convergence)
    unsigned long currentCycle = currentMillis / MASTER_CYCLE;

    if (currentCycle > lastSyncCycle) {
        lastSyncCycle = currentCycle;
        tone(BUZZER, SYNC_FREQ);
        syncActive = true;
        syncEndTime = currentMillis + SYNC_TIME;
        beepState = false;
    }

    if (syncActive) {
        if (currentMillis >= syncEndTime) {
            noTone(BUZZER);
            syncActive = false;
            beepTimer = currentMillis;
            beepState = false;
            currentBuzzerState = -1;
        }
        return;
    }
}

```

6. CONCLUSION

The Traffic Signal Synchronization System provides an intuitive, practical bridge between abstract number theory and urban infrastructure engineering. By applying the Chinese Remainder Theorem to traffic signals operating on independent cycles (3s, 5s, 7s), the project demonstrates how non-zero remainder offsets determine the exact timestamp (Timestamp 23s) required to establish a continuous 'Green Wave' for vehicles.

Both the physical Arduino Nano hardware prototype and the digital twin web application successfully visualize modular congruences, offering an accessible learning tool that highlights the real-world power of modular arithmetic in signal timing, telecommunications, and digital computing.

7. INDIVIDUAL CONTRIBUTION

Name: Jose Alex

Class: S3 CSE B

Roll No: 55

Role: Mathematical Proofs & Solution Verification

Contribution Summary:

I worked on double-checking the math steps and making sure our equations were completely correct. Working with Joel Duke and Jyothika, I spent time verifying that 105 is the true repeat cycle for 3, 5, and 7. I worked through the step-by-step modulo inverse math by hand to confirm $y_1=2$, $y_2=1$, and $y_3=1$, and tested edge cases to make sure there were no calculation errors in our written report or website explanations.

MATHEMATICS PROJECT ASSIGNMENT REPORT

PROJECT TITLE: Traffic Signal Synchronization System Using Chinese Remainder Theorem	STUDENT NAME: Joseph Alex
COURSE: Number Theory & Discrete Mathematics Application	ROLL NO: 56
CLASS: S3 CSE B	ROLE / FUNCTION: Hardware Sourcing & Build Testing

1. ABSTRACT

The Chinese Remainder Theorem (CRT) is a fundamental result in number theory that determines a unique integer solution x for a system of linear congruences with pairwise coprime moduli. While often taught abstractly, CRT has direct applications in urban traffic signal synchronization, digital telecommunications, and parallel computer architectures.

This project demonstrates CRT using both a physical working Arduino Nano traffic signal model and an interactive digital twin web application. By modeling traffic signals as modular timing cycles, the project illustrates how independent light timers (3s, 5s, 7s) align to create a continuous 'Green Wave' for vehicles at Minute 23 (105s master recurrence).

2. INTRODUCTION & OBJECTIVES

When traffic signals operate on independent timer cycles along a main avenue, drivers frequently encounter red lights. Traffic engineers solve this by calculating when all signals will simultaneously show green.

Project Objectives:

1. To understand the mathematical principles of the Chinese Remainder Theorem (CRT).
2. To differentiate between Least Common Multiple (LCM) and CRT when remainder offsets are present.
3. To model traffic signal timing cycles (3s, 5s, 7s) as modular congruences.
4. To build a physical working model powered by an Arduino Nano microcontroller.
5. To provide clear visual, audio, and waveform telemetry feedback upon signal synchronization.

3. MATHEMATICAL FORMULATION (CRT vs. LCM)

3.1 The Traffic Signal Equations

Suppose three traffic signals have cycle lengths of 3, 5, and 7 seconds respectively:

- Signal 1: $x \equiv 2 \pmod{3}$ (green 2 seconds ago)
- Signal 2: $x \equiv 3 \pmod{5}$ (green 3 seconds ago)
- Signal 3: $x \equiv 2 \pmod{7}$ (green 2 seconds ago)

3.2 Why Simple LCM Cannot Solve This Problem

- LCM: Used only when all signals start at Minute 0 with zero remainder offset ($x \equiv 0$).
- CRT: Required when remainder offsets (2, 3, 2) are distinct and non-zero.

3.3 CRT Solution Procedure

1. Total Product: $M = 3 \times 5 \times 7 = 105$.
2. Partial Products: $M_1 = 35$, $M_2 = 21$, $M_3 = 15$.

3. Modular Inverses:

- $35 * y1 \equiv 1 \pmod{3} \Rightarrow 2 * y1 \equiv 1 \pmod{3} \Rightarrow y1 = 2$
- $21 * y2 \equiv 1 \pmod{5} \Rightarrow 1 * y2 \equiv 1 \pmod{5} \Rightarrow y2 = 1$
- $15 * y3 \equiv 1 \pmod{7} \Rightarrow 1 * y3 \equiv 1 \pmod{7} \Rightarrow y3 = 1$

4. Compute x:

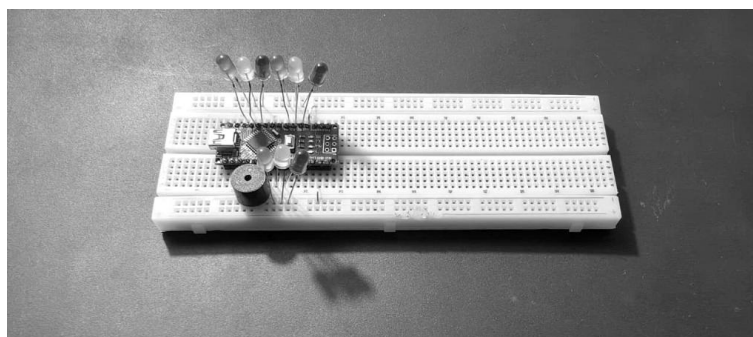
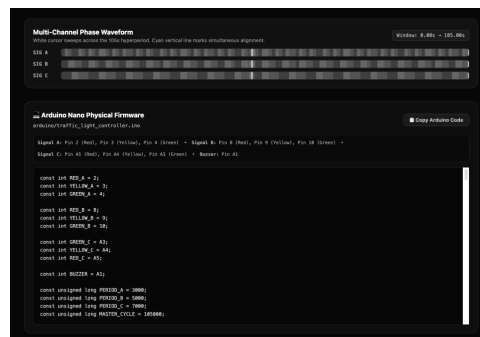
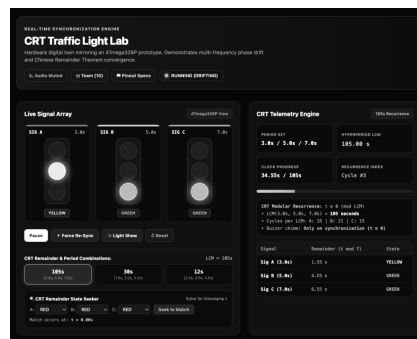
- $x = (2 \times 35 \times 2) + (3 \times 21 \times 1) + (2 \times 15 \times 1)$
- $x = 140 + 63 + 30 = 233$
- $x \equiv 233 \pmod{105} \equiv 23 \pmod{105}$

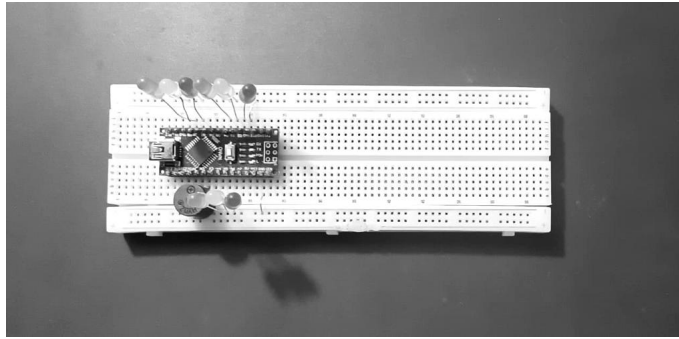
The unique green-wave solution within $[0, 104]$ is Timestamp 23s.

4. PHYSICAL HARDWARE & DIGITAL TWIN IMPLEMENTATION

The hardware prototype is driven by an Arduino Nano microcontroller controlling a 3-intersection LED traffic array and piezo buzzer. The physical build is paired with an interactive digital twin web application accessible at:

■ Interactive Web App & Project Repository: <https://github.com/jvstin47/CRT--Chinese-Reminder-Theorem>
(Local Dev Server: <http://localhost:5173/>)





Interactive Hardware & Digital Twin Controls:

1. Live Signal Array (SIG A 3.0s, SIG B 5.0s, SIG C 7.0s): Monitors real-time LED states across intersections.
 2. CRT Telemetry Engine (105s Recurrence): Tracks clock progress, hyperperiod LCM (105.00s), and recurrence index.
 3. Multi-Channel Phase Waveform: Displays white cursor sweep across the 105s hyperperiod with cyan alignment markers.
 4. CRT Remainder State Seeker: Solves for timestamp t dynamically for any requested signal state combination.
-

5. HARDWARE SOURCE CODE (ARDUINO NANO FIRMWARE - C++)

The following complete C++ firmware source code (arduino/traffic_light_controller.ino) is executed directly on the Arduino Nano microcontroller to drive the physical traffic light hardware:

```
// Arduino Nano Physical Traffic Light Controller
// File: arduino/traffic_light_controller.ino

const int RED_A = 2;
const int YELLOW_A = 3;
const int GREEN_A = 4;

const int RED_B = 8;
const int YELLOW_B = 9;
const int GREEN_B = 10;

const int GREEN_C = A3;
const int YELLOW_C = A4;
const int RED_C = A5;

const int BUZZER = A1;

// Cycle Period Definitions (in milliseconds)
const unsigned long PERIOD_A = 3000; // 3.0s Signal Cycle
const unsigned long PERIOD_B = 5000; // 5.0s Signal Cycle
const unsigned long PERIOD_C = 7000; // 7.0s Signal Cycle
const unsigned long MASTER_CYCLE = 105000; // 105.0s Hyperperiod LCM

const int RED_FREQ = 400;
const int YELLOW_FREQ = 600;
const int GREEN_FREQ = 800;
const int SYNC_FREQ = 1000;

const unsigned long BEEP_ON_TIME = 150;
const unsigned long BEEP_OFF_TIME = 350;
const unsigned long SYNC_TIME = 600;

unsigned long lastMillis = 0;
unsigned long lastSyncCycle = 0;
unsigned long beepTimer = 0;
unsigned long syncEndTime = 0;
```

```

bool beepState = false;
bool syncActive = false;
int currentBuzzerState = -1;

void setup() {
    pinMode(RED_A, OUTPUT);
    pinMode(YELLOW_A, OUTPUT);
    pinMode(GREEN_A, OUTPUT);

    pinMode(RED_B, OUTPUT);
    pinMode(YELLOW_B, OUTPUT);
    pinMode(GREEN_B, OUTPUT);

    pinMode(RED_C, OUTPUT);
    pinMode(YELLOW_C, OUTPUT);
    pinMode(GREEN_C, OUTPUT);

    pinMode(BUZZER, OUTPUT);

    setAllRed();

    tone(BUZZER, SYNC_FREQ);
    syncActive = true;
    syncEndTime = millis() + SYNC_TIME;

    lastMillis = millis();
    beepTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    // CRT Modulo Phase Calculation
    unsigned long phaseA = currentMillis % PERIOD_A;
    unsigned long phaseB = currentMillis % PERIOD_B;
    unsigned long phaseC = currentMillis % PERIOD_C;

    int stateA = updateLightA(phaseA);
    int stateB = updateLightB(phaseB);
    int stateC = updateLightC(phaseC);

    // Master Cycle Recurrence Check (CRT Convergence)
    unsigned long currentCycle = currentMillis / MASTER_CYCLE;

    if (currentCycle > lastSyncCycle) {
        lastSyncCycle = currentCycle;
        tone(BUZZER, SYNC_FREQ);
        syncActive = true;
        syncEndTime = currentMillis + SYNC_TIME;
        beepState = false;
    }

    if (syncActive) {
        if (currentMillis >= syncEndTime) {
            noTone(BUZZER);
            syncActive = false;
            beepTimer = currentMillis;
            beepState = false;
            currentBuzzerState = -1;
        }
        return;
    }
}

```

6. CONCLUSION

The Traffic Signal Synchronization System provides an intuitive, practical bridge between abstract number theory and urban infrastructure engineering. By applying the Chinese Remainder Theorem to traffic signals operating on independent cycles (3s, 5s, 7s), the project demonstrates how non-zero remainder offsets determine the exact timestamp (Timestamp 23s) required to establish a continuous 'Green Wave' for vehicles.

Both the physical Arduino Nano hardware prototype and the digital twin web application successfully visualize modular congruences, offering an accessible learning tool that highlights the real-world power of modular arithmetic in signal timing, telecommunications, and digital computing.

7. INDIVIDUAL CONTRIBUTION

Name: Joseph Alex

Class: S3 CSE B

Roll No: 56

Role: Hardware Sourcing & Build Testing

Contribution Summary:

I handled getting the physical circuit parts and testing the board. Working with Joseph J and Justin, I bought the Arduino Nano board, breadboards, resistors, red/yellow/green LEDs, and the buzzer. I spent time plugging in wires on the breadboard, checking if the voltage levels were correct, and making sure no wires were loose. I ran test runs with the circuit plugged into a laptop to make sure all LEDs turned on reliably.

MATHEMATICS PROJECT ASSIGNMENT REPORT

PROJECT TITLE: Traffic Signal Synchronization System Using Chinese Remainder Theorem	STUDENT NAME: Joseph J
COURSE: Number Theory & Discrete Mathematics Application	ROLL NO: 57
CLASS: S3 CSE B	ROLE / FUNCTION: Component Procurement & Application Ideas

1. ABSTRACT

The Chinese Remainder Theorem (CRT) is a fundamental result in number theory that determines a unique integer solution x for a system of linear congruences with pairwise coprime moduli. While often taught abstractly, CRT has direct applications in urban traffic signal synchronization, digital telecommunications, and parallel computer architectures.

This project demonstrates CRT using both a physical working Arduino Nano traffic signal model and an interactive digital twin web application. By modeling traffic signals as modular timing cycles, the project illustrates how independent light timers (3s, 5s, 7s) align to create a continuous 'Green Wave' for vehicles at Minute 23 (105s master recurrence).

2. INTRODUCTION & OBJECTIVES

When traffic signals operate on independent timer cycles along a main avenue, drivers frequently encounter red lights. Traffic engineers solve this by calculating when all signals will simultaneously show green.

Project Objectives:

1. To understand the mathematical principles of the Chinese Remainder Theorem (CRT).
2. To differentiate between Least Common Multiple (LCM) and CRT when remainder offsets are present.
3. To model traffic signal timing cycles (3s, 5s, 7s) as modular congruences.
4. To build a physical working model powered by an Arduino Nano microcontroller.
5. To provide clear visual, audio, and waveform telemetry feedback upon signal synchronization.

3. MATHEMATICAL FORMULATION (CRT vs. LCM)

3.1 The Traffic Signal Equations

Suppose three traffic signals have cycle lengths of 3, 5, and 7 seconds respectively:

- Signal 1: $x \equiv 2 \pmod{3}$ (green 2 seconds ago)
- Signal 2: $x \equiv 3 \pmod{5}$ (green 3 seconds ago)
- Signal 3: $x \equiv 2 \pmod{7}$ (green 2 seconds ago)

3.2 Why Simple LCM Cannot Solve This Problem

- LCM: Used only when all signals start at Minute 0 with zero remainder offset ($x \equiv 0$).
- CRT: Required when remainder offsets (2, 3, 2) are distinct and non-zero.

3.3 CRT Solution Procedure

1. Total Product: $M = 3 \times 5 \times 7 = 105$.

2. Partial Products: $M1 = 35$, $M2 = 21$, $M3 = 15$.

3. Modular Inverses:

- $35 * y1 \equiv 1 \pmod{3} \Rightarrow 2 * y1 \equiv 1 \pmod{3} \Rightarrow y1 = 2$
- $21 * y2 \equiv 1 \pmod{5} \Rightarrow 1 * y2 \equiv 1 \pmod{5} \Rightarrow y2 = 1$
- $15 * y3 \equiv 1 \pmod{7} \Rightarrow 1 * y3 \equiv 1 \pmod{7} \Rightarrow y3 = 1$

4. Compute x :

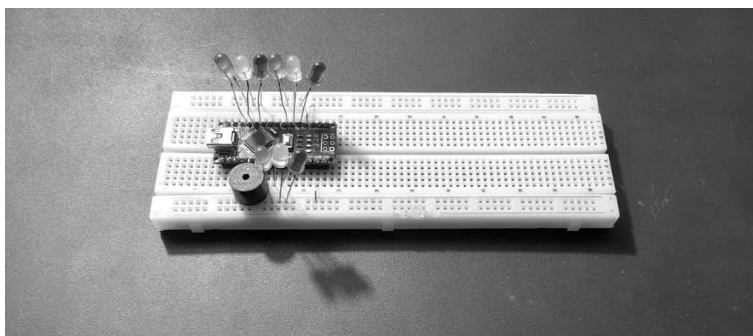
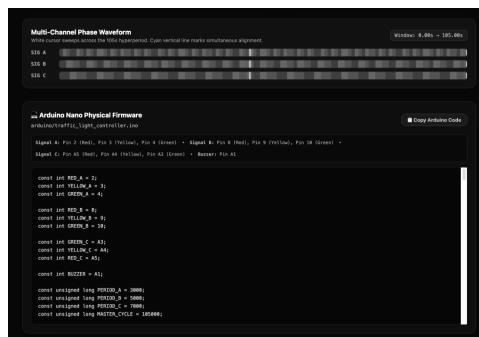
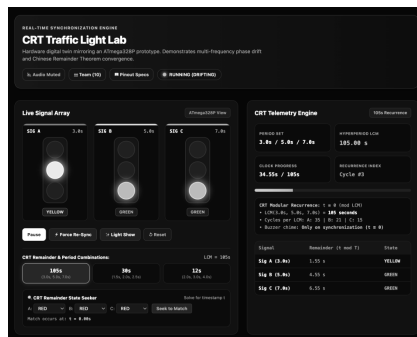
- $x = (2 \times 35 \times 2) + (3 \times 21 \times 1) + (2 \times 15 \times 1)$
- $x = 140 + 63 + 30 = 233$
- $x \equiv 233 \pmod{105} \equiv 23 \pmod{105}$

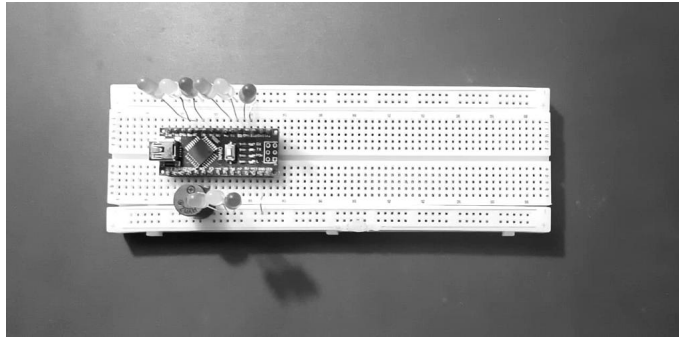
The unique green-wave solution within $[0, 104]$ is Timestamp 23s.

4. PHYSICAL HARDWARE & DIGITAL TWIN IMPLEMENTATION

The hardware prototype is driven by an Arduino Nano microcontroller controlling a 3-intersection LED traffic array and piezo buzzer. The physical build is paired with an interactive digital twin web application accessible at:

■ Interactive Web App & Project Repository: <https://github.com/jvstin47/CRT--Chinese-Reminder-Theorem>
(Local Dev Server: <http://localhost:5173/>)





Interactive Hardware & Digital Twin Controls:

1. Live Signal Array (SIG A 3.0s, SIG B 5.0s, SIG C 7.0s): Monitors real-time LED states across intersections.
 2. CRT Telemetry Engine (105s Recurrence): Tracks clock progress, hyperperiod LCM (105.00s), and recurrence index.
 3. Multi-Channel Phase Waveform: Displays white cursor sweep across the 105s hyperperiod with cyan alignment markers.
 4. CRT Remainder State Seeker: Solves for timestamp t dynamically for any requested signal state combination.
-

5. HARDWARE SOURCE CODE (ARDUINO NANO FIRMWARE - C++)

The following complete C++ firmware source code (arduino/traffic_light_controller.ino) is executed directly on the Arduino Nano microcontroller to drive the physical traffic light hardware:

```
// Arduino Nano Physical Traffic Light Controller
// File: arduino/traffic_light_controller.ino

const int RED_A = 2;
const int YELLOW_A = 3;
const int GREEN_A = 4;

const int RED_B = 8;
const int YELLOW_B = 9;
const int GREEN_B = 10;

const int GREEN_C = A3;
const int YELLOW_C = A4;
const int RED_C = A5;

const int BUZZER = A1;

// Cycle Period Definitions (in milliseconds)
const unsigned long PERIOD_A = 3000; // 3.0s Signal Cycle
const unsigned long PERIOD_B = 5000; // 5.0s Signal Cycle
const unsigned long PERIOD_C = 7000; // 7.0s Signal Cycle
const unsigned long MASTER_CYCLE = 105000; // 105.0s Hyperperiod LCM

const int RED_FREQ = 400;
const int YELLOW_FREQ = 600;
const int GREEN_FREQ = 800;
const int SYNC_FREQ = 1000;

const unsigned long BEEP_ON_TIME = 150;
const unsigned long BEEP_OFF_TIME = 350;
const unsigned long SYNC_TIME = 600;

unsigned long lastMillis = 0;
unsigned long lastSyncCycle = 0;
unsigned long beepTimer = 0;
unsigned long syncEndTime = 0;
```

```

bool beepState = false;
bool syncActive = false;
int currentBuzzerState = -1;

void setup() {
    pinMode(RED_A, OUTPUT);
    pinMode(YELLOW_A, OUTPUT);
    pinMode(GREEN_A, OUTPUT);

    pinMode(RED_B, OUTPUT);
    pinMode(YELLOW_B, OUTPUT);
    pinMode(GREEN_B, OUTPUT);

    pinMode(RED_C, OUTPUT);
    pinMode(YELLOW_C, OUTPUT);
    pinMode(GREEN_C, OUTPUT);

    pinMode(BUZZER, OUTPUT);

    setAllRed();

    tone(BUZZER, SYNC_FREQ);
    syncActive = true;
    syncEndTime = millis() + SYNC_TIME;

    lastMillis = millis();
    beepTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    // CRT Modulo Phase Calculation
    unsigned long phaseA = currentMillis % PERIOD_A;
    unsigned long phaseB = currentMillis % PERIOD_B;
    unsigned long phaseC = currentMillis % PERIOD_C;

    int stateA = updateLightA(phaseA);
    int stateB = updateLightB(phaseB);
    int stateC = updateLightC(phaseC);

    // Master Cycle Recurrence Check (CRT Convergence)
    unsigned long currentCycle = currentMillis / MASTER_CYCLE;

    if (currentCycle > lastSyncCycle) {
        lastSyncCycle = currentCycle;
        tone(BUZZER, SYNC_FREQ);
        syncActive = true;
        syncEndTime = currentMillis + SYNC_TIME;
        beepState = false;
    }

    if (syncActive) {
        if (currentMillis >= syncEndTime) {
            noTone(BUZZER);
            syncActive = false;
            beepTimer = currentMillis;
            beepState = false;
            currentBuzzerState = -1;
        }
        return;
    }
}

```

6. CONCLUSION

The Traffic Signal Synchronization System provides an intuitive, practical bridge between abstract number theory and urban infrastructure engineering. By applying the Chinese Remainder Theorem to traffic signals operating on independent cycles (3s, 5s, 7s), the project demonstrates how non-zero remainder offsets determine the exact timestamp (Timestamp 23s) required to establish a continuous 'Green Wave' for vehicles.

Both the physical Arduino Nano hardware prototype and the digital twin web application successfully visualize modular congruences, offering an accessible learning tool that highlights the real-world power of modular arithmetic in signal timing, telecommunications, and digital computing.

7. INDIVIDUAL CONTRIBUTION

Name: Joseph J

Class: S3 CSE B

Roll No: 57

Role: Component Procurement & Application Ideas

Contribution Summary:

I helped with buying electronic components and researching real-world examples for our project. I worked with Joseph Alex to test the breadboard layout and LED connections. After getting the circuit running, I spent time researching how this same math is used in real life, like synchronizing real traffic signals in cities, sending multiple signals through fiber-optic cables, and timing computer processors.

MATHEMATICS PROJECT ASSIGNMENT REPORT

PROJECT TITLE: Traffic Signal Synchronization System Using Chinese Remainder Theorem	STUDENT NAME: Justin Joe Mathew
COURSE: Number Theory & Discrete Mathematics Application	ROLL NO: 58
CLASS: S3 CSE B	ROLE / FUNCTION: Team Lead & Working Model Engineer

1. ABSTRACT

The Chinese Remainder Theorem (CRT) is a fundamental result in number theory that determines a unique integer solution x for a system of linear congruences with pairwise coprime moduli. While often taught abstractly, CRT has direct applications in urban traffic signal synchronization, digital telecommunications, and parallel computer architectures.

This project demonstrates CRT using both a physical working Arduino Nano traffic signal model and an interactive digital twin web application. By modeling traffic signals as modular timing cycles, the project illustrates how independent light timers (3s, 5s, 7s) align to create a continuous 'Green Wave' for vehicles at Minute 23 (105s master recurrence).

2. INTRODUCTION & OBJECTIVES

When traffic signals operate on independent timer cycles along a main avenue, drivers frequently encounter red lights. Traffic engineers solve this by calculating when all signals will simultaneously show green.

Project Objectives:

1. To understand the mathematical principles of the Chinese Remainder Theorem (CRT).
2. To differentiate between Least Common Multiple (LCM) and CRT when remainder offsets are present.
3. To model traffic signal timing cycles (3s, 5s, 7s) as modular congruences.
4. To build a physical working model powered by an Arduino Nano microcontroller.
5. To provide clear visual, audio, and waveform telemetry feedback upon signal synchronization.

3. MATHEMATICAL FORMULATION (CRT vs. LCM)

3.1 The Traffic Signal Equations

Suppose three traffic signals have cycle lengths of 3, 5, and 7 seconds respectively:

- Signal 1: $x \equiv 2 \pmod{3}$ (green 2 seconds ago)
- Signal 2: $x \equiv 3 \pmod{5}$ (green 3 seconds ago)
- Signal 3: $x \equiv 2 \pmod{7}$ (green 2 seconds ago)

3.2 Why Simple LCM Cannot Solve This Problem

- LCM: Used only when all signals start at Minute 0 with zero remainder offset ($x \equiv 0$).
- CRT: Required when remainder offsets (2, 3, 2) are distinct and non-zero.

3.3 CRT Solution Procedure

1. Total Product: $M = 3 \times 5 \times 7 = 105$.

2. Partial Products: $M1 = 35$, $M2 = 21$, $M3 = 15$.

3. Modular Inverses:

- $35 * y1 \equiv 1 \pmod{3} \Rightarrow 2 * y1 \equiv 1 \pmod{3} \Rightarrow y1 = 2$
- $21 * y2 \equiv 1 \pmod{5} \Rightarrow 1 * y2 \equiv 1 \pmod{5} \Rightarrow y2 = 1$
- $15 * y3 \equiv 1 \pmod{7} \Rightarrow 1 * y3 \equiv 1 \pmod{7} \Rightarrow y3 = 1$

4. Compute x :

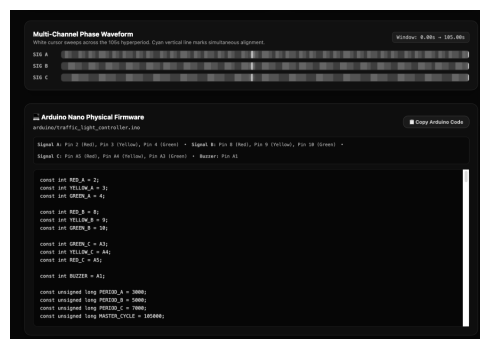
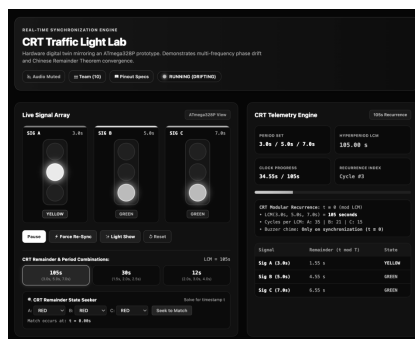
- $x = (2 \times 35 \times 2) + (3 \times 21 \times 1) + (2 \times 15 \times 1)$
- $x = 140 + 63 + 30 = 233$
- $x \equiv 233 \pmod{105} \equiv 23 \pmod{105}$

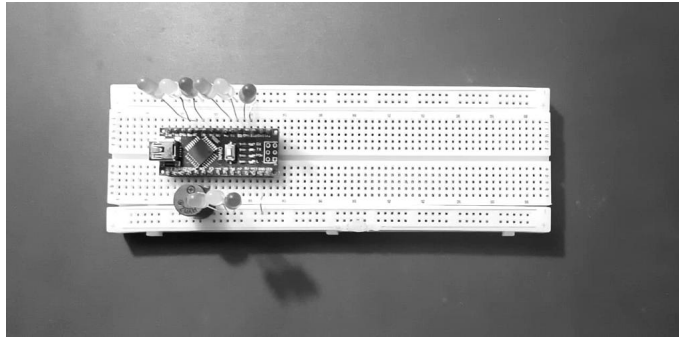
The unique green-wave solution within $[0, 104]$ is Timestamp 23s.

4. PHYSICAL HARDWARE & DIGITAL TWIN IMPLEMENTATION

The hardware prototype is driven by an Arduino Nano microcontroller controlling a 3-intersection LED traffic array and piezo buzzer. The physical build is paired with an interactive digital twin web application accessible at:

■ Interactive Web App & Project Repository: <https://github.com/jvstin47/CRT--Chinese-Reminder-Theorem>
(Local Dev Server: <http://localhost:5173/>)





Interactive Hardware & Digital Twin Controls:

1. Live Signal Array (SIG A 3.0s, SIG B 5.0s, SIG C 7.0s): Monitors real-time LED states across intersections.
2. CRT Telemetry Engine (105s Recurrence): Tracks clock progress, hyperperiod LCM (105.00s), and recurrence index.
3. Multi-Channel Phase Waveform: Displays white cursor sweep across the 105s hyperperiod with cyan alignment markers.
4. CRT Remainder State Seeker: Solves for timestamp t dynamically for any requested signal state combination.

5. HARDWARE SOURCE CODE (ARDUINO NANO FIRMWARE - C++)

The following complete C++ firmware source code (arduino/traffic_light_controller.ino) is executed directly on the Arduino Nano microcontroller to drive the physical traffic light hardware:

```
// Arduino Nano Physical Traffic Light Controller
// File: arduino/traffic_light_controller.ino

const int RED_A = 2;
const int YELLOW_A = 3;
const int GREEN_A = 4;

const int RED_B = 8;
const int YELLOW_B = 9;
const int GREEN_B = 10;

const int GREEN_C = A3;
const int YELLOW_C = A4;
const int RED_C = A5;

const int BUZZER = A1;

// Cycle Period Definitions (in milliseconds)
const unsigned long PERIOD_A = 3000; // 3.0s Signal Cycle
const unsigned long PERIOD_B = 5000; // 5.0s Signal Cycle
const unsigned long PERIOD_C = 7000; // 7.0s Signal Cycle
const unsigned long MASTER_CYCLE = 105000; // 105.0s Hyperperiod LCM

const int RED_FREQ = 400;
const int YELLOW_FREQ = 600;
const int GREEN_FREQ = 800;
const int SYNC_FREQ = 1000;

const unsigned long BEEP_ON_TIME = 150;
const unsigned long BEEP_OFF_TIME = 350;
const unsigned long SYNC_TIME = 600;

unsigned long lastMillis = 0;
unsigned long lastSyncCycle = 0;
unsigned long beepTimer = 0;
unsigned long syncEndTime = 0;
```

```

bool beepState = false;
bool syncActive = false;
int currentBuzzerState = -1;

void setup() {
    pinMode(RED_A, OUTPUT);
    pinMode(YELLOW_A, OUTPUT);
    pinMode(GREEN_A, OUTPUT);

    pinMode(RED_B, OUTPUT);
    pinMode(YELLOW_B, OUTPUT);
    pinMode(GREEN_B, OUTPUT);

    pinMode(RED_C, OUTPUT);
    pinMode(YELLOW_C, OUTPUT);
    pinMode(GREEN_C, OUTPUT);

    pinMode(BUZZER, OUTPUT);

    setAllRed();

    tone(BUZZER, SYNC_FREQ);
    syncActive = true;
    syncEndTime = millis() + SYNC_TIME;

    lastMillis = millis();
    beepTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    // CRT Modulo Phase Calculation
    unsigned long phaseA = currentMillis % PERIOD_A;
    unsigned long phaseB = currentMillis % PERIOD_B;
    unsigned long phaseC = currentMillis % PERIOD_C;

    int stateA = updateLightA(phaseA);
    int stateB = updateLightB(phaseB);
    int stateC = updateLightC(phaseC);

    // Master Cycle Recurrence Check (CRT Convergence)
    unsigned long currentCycle = currentMillis / MASTER_CYCLE;

    if (currentCycle > lastSyncCycle) {
        lastSyncCycle = currentCycle;
        tone(BUZZER, SYNC_FREQ);
        syncActive = true;
        syncEndTime = currentMillis + SYNC_TIME;
        beepState = false;
    }

    if (syncActive) {
        if (currentMillis >= syncEndTime) {
            noTone(BUZZER);
            syncActive = false;
            beepTimer = currentMillis;
            beepState = false;
            currentBuzzerState = -1;
        }
        return;
    }
}

```

6. CONCLUSION

The Traffic Signal Synchronization System provides an intuitive, practical bridge between abstract number theory and urban infrastructure engineering. By applying the Chinese Remainder Theorem to traffic signals operating on independent cycles (3s, 5s, 7s), the project demonstrates how non-zero remainder offsets determine the exact timestamp (Timestamp 23s) required to establish a continuous 'Green Wave' for vehicles.

Both the physical Arduino Nano hardware prototype and the digital twin web application successfully visualize modular congruences, offering an accessible learning tool that highlights the real-world power of modular arithmetic in signal timing, telecommunications, and digital computing.

7. INDIVIDUAL CONTRIBUTION

Name: Justin Joe Mathew

Class: S3 CSE B

Roll No: 58

Role: Team Lead & Working Model Engineer

Contribution Summary:

As team lead, I helped organize group work and oversaw the project build. I spent time building the physical 3-light breadboard model and wrote the Arduino C++ code (`traffic_light_controller.ino`) to run the light timers and buzzer sound. Working with Jyothis on hardware wiring and Johaan on the website, I made sure the physical board and website both showed the exact same traffic light timing.

MATHEMATICS PROJECT ASSIGNMENT REPORT

PROJECT TITLE: Traffic Signal Synchronization System Using Chinese Remainder Theorem	STUDENT NAME: Jyothika Prakash
COURSE: Number Theory & Discrete Mathematics Application	ROLL NO: 59
CLASS: S3 CSE B	ROLE / FUNCTION: Pedagogical Systems & Educational Design

1. ABSTRACT

The Chinese Remainder Theorem (CRT) is a fundamental result in number theory that determines a unique integer solution x for a system of linear congruences with pairwise coprime moduli. While often taught abstractly, CRT has direct applications in urban traffic signal synchronization, digital telecommunications, and parallel computer architectures.

This project demonstrates CRT using both a physical working Arduino Nano traffic signal model and an interactive digital twin web application. By modeling traffic signals as modular timing cycles, the project illustrates how independent light timers (3s, 5s, 7s) align to create a continuous 'Green Wave' for vehicles at Minute 23 (105s master recurrence).

2. INTRODUCTION & OBJECTIVES

When traffic signals operate on independent timer cycles along a main avenue, drivers frequently encounter red lights. Traffic engineers solve this by calculating when all signals will simultaneously show green.

Project Objectives:

1. To understand the mathematical principles of the Chinese Remainder Theorem (CRT).
2. To differentiate between Least Common Multiple (LCM) and CRT when remainder offsets are present.
3. To model traffic signal timing cycles (3s, 5s, 7s) as modular congruences.
4. To build a physical working model powered by an Arduino Nano microcontroller.
5. To provide clear visual, audio, and waveform telemetry feedback upon signal synchronization.

3. MATHEMATICAL FORMULATION (CRT vs. LCM)

3.1 The Traffic Signal Equations

Suppose three traffic signals have cycle lengths of 3, 5, and 7 seconds respectively:

- Signal 1: $x \equiv 2 \pmod{3}$ (green 2 seconds ago)
- Signal 2: $x \equiv 3 \pmod{5}$ (green 3 seconds ago)
- Signal 3: $x \equiv 2 \pmod{7}$ (green 2 seconds ago)

3.2 Why Simple LCM Cannot Solve This Problem

- LCM: Used only when all signals start at Minute 0 with zero remainder offset ($x \equiv 0$).
- CRT: Required when remainder offsets (2, 3, 2) are distinct and non-zero.

3.3 CRT Solution Procedure

1. Total Product: $M = 3 \times 5 \times 7 = 105$.

2. Partial Products: $M1 = 35$, $M2 = 21$, $M3 = 15$.

3. Modular Inverses:

- $35 * y1 \equiv 1 \pmod{3} \Rightarrow 2 * y1 \equiv 1 \pmod{3} \Rightarrow y1 = 2$
- $21 * y2 \equiv 1 \pmod{5} \Rightarrow 1 * y2 \equiv 1 \pmod{5} \Rightarrow y2 = 1$
- $15 * y3 \equiv 1 \pmod{7} \Rightarrow 1 * y3 \equiv 1 \pmod{7} \Rightarrow y3 = 1$

4. Compute x :

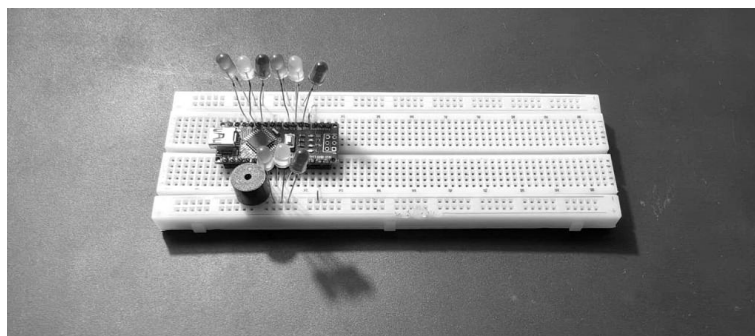
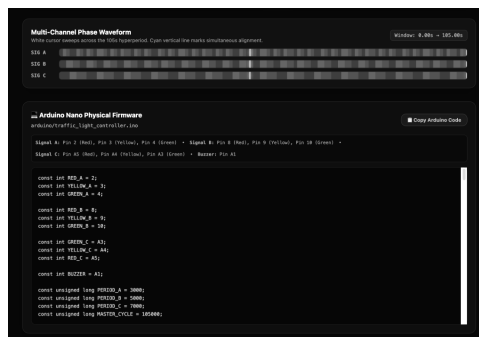
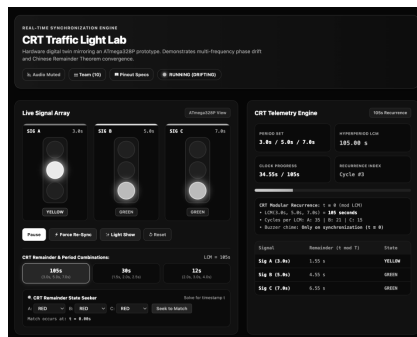
- $x = (2 \times 35 \times 2) + (3 \times 21 \times 1) + (2 \times 15 \times 1)$
- $x = 140 + 63 + 30 = 233$
- $x \equiv 233 \pmod{105} \equiv 23 \pmod{105}$

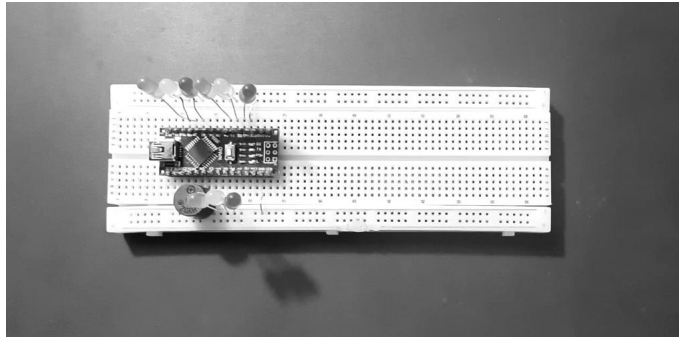
The unique green-wave solution within $[0, 104]$ is Timestamp 23s.

4. PHYSICAL HARDWARE & DIGITAL TWIN IMPLEMENTATION

The hardware prototype is driven by an Arduino Nano microcontroller controlling a 3-intersection LED traffic array and piezo buzzer. The physical build is paired with an interactive digital twin web application accessible at:

■ Interactive Web App & Project Repository: <https://github.com/jvstin47/CRT--Chinese-Reminder-Theorem>
(Local Dev Server: <http://localhost:5173/>)





Interactive Hardware & Digital Twin Controls:

1. Live Signal Array (SIG A 3.0s, SIG B 5.0s, SIG C 7.0s): Monitors real-time LED states across intersections.
 2. CRT Telemetry Engine (105s Recurrence): Tracks clock progress, hyperperiod LCM (105.00s), and recurrence index.
 3. Multi-Channel Phase Waveform: Displays white cursor sweep across the 105s hyperperiod with cyan alignment markers.
 4. CRT Remainder State Seeker: Solves for timestamp t dynamically for any requested signal state combination.
-

5. HARDWARE SOURCE CODE (ARDUINO NANO FIRMWARE - C++)

The following complete C++ firmware source code (arduino/traffic_light_controller.ino) is executed directly on the Arduino Nano microcontroller to drive the physical traffic light hardware:

```
// Arduino Nano Physical Traffic Light Controller
// File: arduino/traffic_light_controller.ino

const int RED_A = 2;
const int YELLOW_A = 3;
const int GREEN_A = 4;

const int RED_B = 8;
const int YELLOW_B = 9;
const int GREEN_B = 10;

const int GREEN_C = A3;
const int YELLOW_C = A4;
const int RED_C = A5;

const int BUZZER = A1;

// Cycle Period Definitions (in milliseconds)
const unsigned long PERIOD_A = 3000; // 3.0s Signal Cycle
const unsigned long PERIOD_B = 5000; // 5.0s Signal Cycle
const unsigned long PERIOD_C = 7000; // 7.0s Signal Cycle
const unsigned long MASTER_CYCLE = 105000; // 105.0s Hyperperiod LCM

const int RED_FREQ = 400;
const int YELLOW_FREQ = 600;
const int GREEN_FREQ = 800;
const int SYNC_FREQ = 1000;

const unsigned long BEEP_ON_TIME = 150;
const unsigned long BEEP_OFF_TIME = 350;
const unsigned long SYNC_TIME = 600;

unsigned long lastMillis = 0;
unsigned long lastSyncCycle = 0;
unsigned long beepTimer = 0;
unsigned long syncEndTime = 0;
```

```

bool beepState = false;
bool syncActive = false;
int currentBuzzerState = -1;

void setup() {
    pinMode(RED_A, OUTPUT);
    pinMode(YELLOW_A, OUTPUT);
    pinMode(GREEN_A, OUTPUT);

    pinMode(RED_B, OUTPUT);
    pinMode(YELLOW_B, OUTPUT);
    pinMode(GREEN_B, OUTPUT);

    pinMode(RED_C, OUTPUT);
    pinMode(YELLOW_C, OUTPUT);
    pinMode(GREEN_C, OUTPUT);

    pinMode(BUZZER, OUTPUT);

    setAllRed();

    tone(BUZZER, SYNC_FREQ);
    syncActive = true;
    syncEndTime = millis() + SYNC_TIME;

    lastMillis = millis();
    beepTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    // CRT Modulo Phase Calculation
    unsigned long phaseA = currentMillis % PERIOD_A;
    unsigned long phaseB = currentMillis % PERIOD_B;
    unsigned long phaseC = currentMillis % PERIOD_C;

    int stateA = updateLightA(phaseA);
    int stateB = updateLightB(phaseB);
    int stateC = updateLightC(phaseC);

    // Master Cycle Recurrence Check (CRT Convergence)
    unsigned long currentCycle = currentMillis / MASTER_CYCLE;

    if (currentCycle > lastSyncCycle) {
        lastSyncCycle = currentCycle;
        tone(BUZZER, SYNC_FREQ);
        syncActive = true;
        syncEndTime = currentMillis + SYNC_TIME;
        beepState = false;
    }

    if (syncActive) {
        if (currentMillis >= syncEndTime) {
            noTone(BUZZER);
            syncActive = false;
            beepTimer = currentMillis;
            beepState = false;
            currentBuzzerState = -1;
        }
        return;
    }
}

```

6. CONCLUSION

The Traffic Signal Synchronization System provides an intuitive, practical bridge between abstract number theory and urban infrastructure engineering. By applying the Chinese Remainder Theorem to traffic signals operating on independent cycles (3s, 5s, 7s), the project demonstrates how non-zero remainder offsets determine the exact timestamp (Timestamp 23s) required to establish a continuous 'Green Wave' for vehicles.

Both the physical Arduino Nano hardware prototype and the digital twin web application successfully visualize modular congruences, offering an accessible learning tool that highlights the real-world power of modular arithmetic in signal timing, telecommunications, and digital computing.

7. INDIVIDUAL CONTRIBUTION

Name: Jyothika Prakash

Class: S3 CSE B

Roll No: 59

Role: Pedagogical Systems & Educational Design

Contribution Summary:

I worked on the educational parts of the website and writing clear explanations. Working with Joel Duke and Jose Alex, I spent time creating the 3-step hint system on the website so users can get clues on how to solve the math puzzle. I wrote simple, easy-to-read guides that explain how traffic light timing relates to modulo remainders without using confusing textbook jargon.

MATHEMATICS PROJECT ASSIGNMENT REPORT

PROJECT TITLE: Traffic Signal Synchronization System Using Chinese Remainder Theorem	STUDENT NAME: Jyothis Liju
COURSE: Number Theory & Discrete Mathematics Application	ROLL NO: 60
CLASS: S3 CSE B	ROLE / FUNCTION: Working Model Co-Developer

1. ABSTRACT

The Chinese Remainder Theorem (CRT) is a fundamental result in number theory that determines a unique integer solution x for a system of linear congruences with pairwise coprime moduli. While often taught abstractly, CRT has direct applications in urban traffic signal synchronization, digital telecommunications, and parallel computer architectures.

This project demonstrates CRT using both a physical working Arduino Nano traffic signal model and an interactive digital twin web application. By modeling traffic signals as modular timing cycles, the project illustrates how independent light timers (3s, 5s, 7s) align to create a continuous 'Green Wave' for vehicles at Minute 23 (105s master recurrence).

2. INTRODUCTION & OBJECTIVES

When traffic signals operate on independent timer cycles along a main avenue, drivers frequently encounter red lights. Traffic engineers solve this by calculating when all signals will simultaneously show green.

Project Objectives:

1. To understand the mathematical principles of the Chinese Remainder Theorem (CRT).
2. To differentiate between Least Common Multiple (LCM) and CRT when remainder offsets are present.
3. To model traffic signal timing cycles (3s, 5s, 7s) as modular congruences.
4. To build a physical working model powered by an Arduino Nano microcontroller.
5. To provide clear visual, audio, and waveform telemetry feedback upon signal synchronization.

3. MATHEMATICAL FORMULATION (CRT vs. LCM)

3.1 The Traffic Signal Equations

Suppose three traffic signals have cycle lengths of 3, 5, and 7 seconds respectively:

- Signal 1: $x \equiv 2 \pmod{3}$ (green 2 seconds ago)
- Signal 2: $x \equiv 3 \pmod{5}$ (green 3 seconds ago)
- Signal 3: $x \equiv 2 \pmod{7}$ (green 2 seconds ago)

3.2 Why Simple LCM Cannot Solve This Problem

- LCM: Used only when all signals start at Minute 0 with zero remainder offset ($x \equiv 0$).
- CRT: Required when remainder offsets (2, 3, 2) are distinct and non-zero.

3.3 CRT Solution Procedure

1. Total Product: $M = 3 \times 5 \times 7 = 105$.
2. Partial Products: $M_1 = 35$, $M_2 = 21$, $M_3 = 15$.

3. Modular Inverses:

- $35 * y1 \equiv 1 \pmod{3} \Rightarrow 2 * y1 \equiv 1 \pmod{3} \Rightarrow y1 = 2$
- $21 * y2 \equiv 1 \pmod{5} \Rightarrow 1 * y2 \equiv 1 \pmod{5} \Rightarrow y2 = 1$
- $15 * y3 \equiv 1 \pmod{7} \Rightarrow 1 * y3 \equiv 1 \pmod{7} \Rightarrow y3 = 1$

4. Compute x:

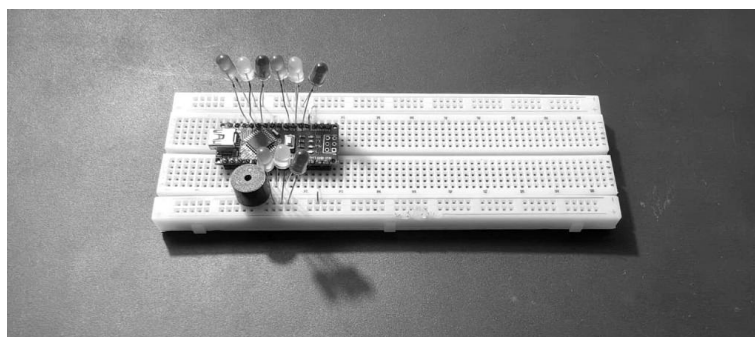
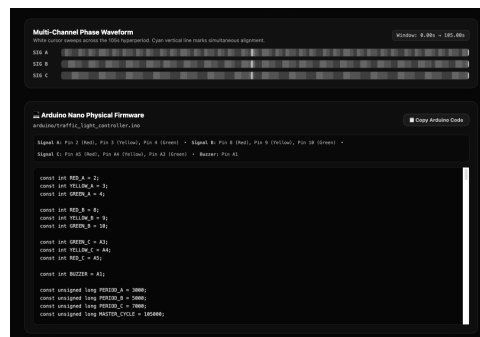
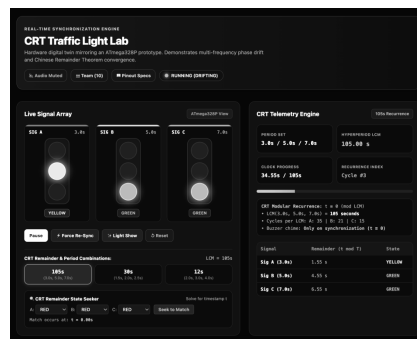
- $x = (2 \times 35 \times 2) + (3 \times 21 \times 1) + (2 \times 15 \times 1)$
- $x = 140 + 63 + 30 = 233$
- $x \equiv 233 \pmod{105} \equiv 23 \pmod{105}$

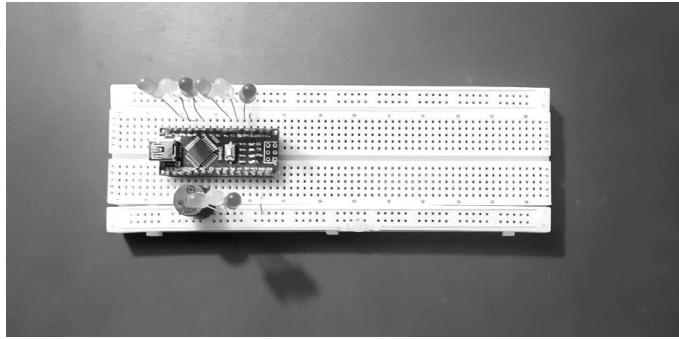
The unique green-wave solution within $[0, 104]$ is Timestamp 23s.

4. PHYSICAL HARDWARE & DIGITAL TWIN IMPLEMENTATION

The hardware prototype is driven by an Arduino Nano microcontroller controlling a 3-intersection LED traffic array and piezo buzzer. The physical build is paired with an interactive digital twin web application accessible at:

■ Interactive Web App & Project Repository: <https://github.com/jvstin47/CRT--Chinese-Reminder-Theorem>
(Local Dev Server: <http://localhost:5173/>)





Interactive Hardware & Digital Twin Controls:

1. Live Signal Array (SIG A 3.0s, SIG B 5.0s, SIG C 7.0s): Monitors real-time LED states across intersections.
2. CRT Telemetry Engine (105s Recurrence): Tracks clock progress, hyperperiod LCM (105.00s), and recurrence index.
3. Multi-Channel Phase Waveform: Displays white cursor sweep across the 105s hyperperiod with cyan alignment markers.
4. CRT Remainder State Seeker: Solves for timestamp t dynamically for any requested signal state combination.

5. HARDWARE SOURCE CODE (ARDUINO NANO FIRMWARE - C++)

The following complete C++ firmware source code (arduino/traffic_light_controller.ino) is executed directly on the Arduino Nano microcontroller to drive the physical traffic light hardware:

```
// Arduino Nano Physical Traffic Light Controller
// File: arduino/traffic_light_controller.ino

const int RED_A = 2;
const int YELLOW_A = 3;
const int GREEN_A = 4;

const int RED_B = 8;
const int YELLOW_B = 9;
const int GREEN_B = 10;

const int GREEN_C = A3;
const int YELLOW_C = A4;
const int RED_C = A5;

const int BUZZER = A1;

// Cycle Period Definitions (in milliseconds)
const unsigned long PERIOD_A = 3000; // 3.0s Signal Cycle
const unsigned long PERIOD_B = 5000; // 5.0s Signal Cycle
const unsigned long PERIOD_C = 7000; // 7.0s Signal Cycle
const unsigned long MASTER_CYCLE = 105000; // 105.0s Hyperperiod LCM

const int RED_FREQ = 400;
const int YELLOW_FREQ = 600;
const int GREEN_FREQ = 800;
const int SYNC_FREQ = 1000;

const unsigned long BEEP_ON_TIME = 150;
const unsigned long BEEP_OFF_TIME = 350;
const unsigned long SYNC_TIME = 600;

unsigned long lastMillis = 0;
unsigned long lastSyncCycle = 0;
unsigned long beepTimer = 0;
unsigned long syncEndTime = 0;
```

```

bool beepState = false;
bool syncActive = false;
int currentBuzzerState = -1;

void setup() {
    pinMode(RED_A, OUTPUT);
    pinMode(YELLOW_A, OUTPUT);
    pinMode(GREEN_A, OUTPUT);

    pinMode(RED_B, OUTPUT);
    pinMode(YELLOW_B, OUTPUT);
    pinMode(GREEN_B, OUTPUT);

    pinMode(RED_C, OUTPUT);
    pinMode(YELLOW_C, OUTPUT);
    pinMode(GREEN_C, OUTPUT);

    pinMode(BUZZER, OUTPUT);

    setAllRed();

    tone(BUZZER, SYNC_FREQ);
    syncActive = true;
    syncEndTime = millis() + SYNC_TIME;

    lastMillis = millis();
    beepTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    // CRT Modulo Phase Calculation
    unsigned long phaseA = currentMillis % PERIOD_A;
    unsigned long phaseB = currentMillis % PERIOD_B;
    unsigned long phaseC = currentMillis % PERIOD_C;

    int stateA = updateLightA(phaseA);
    int stateB = updateLightB(phaseB);
    int stateC = updateLightC(phaseC);

    // Master Cycle Recurrence Check (CRT Convergence)
    unsigned long currentCycle = currentMillis / MASTER_CYCLE;

    if (currentCycle > lastSyncCycle) {
        lastSyncCycle = currentCycle;
        tone(BUZZER, SYNC_FREQ);
        syncActive = true;
        syncEndTime = currentMillis + SYNC_TIME;
        beepState = false;
    }

    if (syncActive) {
        if (currentMillis >= syncEndTime) {
            noTone(BUZZER);
            syncActive = false;
            beepTimer = currentMillis;
            beepState = false;
            currentBuzzerState = -1;
        }
        return;
    }
}

```

6. CONCLUSION

The Traffic Signal Synchronization System provides an intuitive, practical bridge between abstract number theory and urban infrastructure engineering. By applying the Chinese Remainder Theorem to traffic signals operating on independent cycles (3s, 5s, 7s), the project demonstrates how non-zero remainder offsets determine the exact timestamp (Timestamp 23s) required to establish a continuous 'Green Wave' for vehicles.

Both the physical Arduino Nano hardware prototype and the digital twin web application successfully visualize modular congruences, offering an accessible learning tool that highlights the real-world power of modular arithmetic in signal timing, telecommunications, and digital computing.

7. INDIVIDUAL CONTRIBUTION

Name: Jyothis Liju

Class: S3 CSE B

Roll No: 60

Role: Working Model Co-Developer

Contribution Summary:

I co-developed the physical breadboard model, working directly with Justin over two weeks. I helped plug in the LED lights to digital pins 2 through 10 and connected the buzzer to analog pin A1. I tested the circuit while running the Arduino code to check if the LEDs flashed in the correct 3s, 5s, and 7s timing order and made sure the buzzer beeped when all lights turned green at second 23.